

# Documentação Completa — Samsung Xclipse 940 / XO940

**Projeto:** Xclipse Open Project

**Hardware de referência:** Samsung SM-S721B

**Plataforma:** s5e9945 / erd9945 / r12s

**GPU:** Samsung Xclipse 940 / MGFX 147

**Revisão consolidada:** 2026-09-18

## Escopo e regra de leitura

Este é o documento único e completo do XOP. Ele consolida a arquitetura, as evidências, os resultados, os limites e as referências técnicas já descobertas no projeto. O nome correto para a estrutura de hardware que descreve os nós, recursos e dependências da plataforma é **Device Tree**. A palavra “ramificação” não é usada aqui como nome de uma entrega separada.

A documentação distingue rigorosamente entre fato confirmado, evidência estrutural, resultado parcial, indício, hipótese e falha específica. Uma biblioteca presente não é tratada como execução; um ioctl aceito não é tratado como job concluído; e uma operação NNAPI compilada não é confundida com acesso direto ao dispositivo NPU.

## Índice de consolidação

1. Identidade, escopo e política de evidências.
2. Arquitetura completa do SoC, GPU, Device Tree, DRM, memória e execução.
3. Android, Vulkan, OpenCL, loader, namespaces, manifests e segurança.
4. NPU Exynos, NNAPI, ENN, Device Tree NPU, endpoints, permissões e benchmarks.
5. Resultados de validação, erros, bloqueios, variantes e limites de publicação.
6. Capítulos técnicos e relatórios históricos incorporados integralmente.

## Resumo arquitetural consolidado

```
Exynos / Android
├── Device Tree e plataforma
│   ├── SGPU/G3D: sgpu@22200000
│   ├── NPU: npu_exynos
│   ├── power domains, clocks, DVFS, thermal, AFM e reset
│   ├── IRQ, reserved memory, DMA heaps, BTS e IOMMU/SysMMU
│   └── variantes de placa, revisão e firmware
├── GPU Xclipse 940
│   ├── DRM/UAPI e render nodes
│   ├── GEM, BO, TTM, DMA-BUF e memória
│   ├── VM, PTE, PDE, VA, IOMMU e flush
│   ├── contextos, BO_LIST, CS, chunks e IBs
│   ├── scheduler, rings, doorbell, GFX, COMPUTE e SDMA
│   ├── firmware, reset, recovery, fences e syncobjs
│   └── shader, ISA, compiler, texturas e rendering
├── Pilha Android
│   ├── SurfaceFlinger, RenderThread e Gralloc
│   ├── Vulkan/OpenGL/OpenCL e bibliotecas vendor
│   ├── loader, ICD, manifests e HAL
│   ├── linker namespaces e SELinux
│   └── permissões, buffers protegidos e storage noexec
└── NPU Exynos
    ├── NNAPI → HAL ENN → compiler/runtime → NPU
    ├── IDevice/enn e serviço neuralnetworks
    ├── npu_exynos, IOMMU, DVFS, AFM e /dev/vertex10
    ├── operação SOFTMAX rootless confirmada
    └── endpoint vendor direto e ioctls diretos bloqueados/inconclusivos
```

## Capítulo incorporado: 00-project-scope.md

### Escopo e terminologia

#### Finalidade

O Xclipse Open Project documenta o contrato hardware/software das GPUs Samsung Xclipse para que desenvolvedores possam estudar a plataforma com evidência reproduzível. O escopo começa por observação, inventário e mapas técnicos.

Implementações só são consideradas quando o contrato, a licença e a recuperação estiverem suficientemente documentados.

O alvo de referência é a Xclipse 940 associada ao Samsung SM-S721B. XO940 é o nome interno do projeto de documentação. Revisões e gerações futuras devem permanecer separadas até que a compatibilidade seja demonstrada.

#### Vocabulário técnico

A **Device Tree** é a descrição do kernel para regiões de registradores, interrupções, domínios de energia, clocks, DMA, coerência, relações de IOMMU, dependências de reset, strings `compatible` e propriedades de revisão. Ela não é a árvore de diretórios do repositório e não é o driver completo.

O trabalho combina documentação de GPU, bring-up de DRM, análise de memória, observação de filas, investigação de ISA, compiler, integração Vulkan e validação entre revisões. Um componente chamado loader, ICD, layer ou backend deve ser tratado como uma camada distinta até que sua função seja observada diretamente.

## Incluído

O projeto cobre identificação de hardware, revisões de chip, Device Tree, integração de plataforma, DRM, UAPI, GEM, BO, DMA-BUF, IOMMU, memória virtual, faults, scheduler, fences, sync objects, reset, filas GFX/compute/DMA, referências de firmware, clocks, energia, OPP, comportamento térmico, namespaces Android, manifests ICD, SELinux, HWC, packets, registradores, shaders, ISA preliminar, compiler, mapeamento Vulkan, layers de diagnóstico e validação entre revisões.

## Não concluído

Enumeração de API, criação de objetos, retorno de `VK_SUCCESS`, existência de biblioteca, compilação de teste ou presença de uma extensão não constituem execução GPU. Compatibilidade com jogos, WSI, apresentação, todos os formatos, todas as filas e todas as revisões permanece fora de qualquer afirmação automática.

## Regra de evidência

| Artefato ou ação       | Pode estabelecer                                        | Não estabelece sozinho                          |
|------------------------|---------------------------------------------------------|-------------------------------------------------|
| Inicialização de teste | Ambiente preparado.                                     | Execução de GPU ou resultado correto.           |
| Enumeração Vulkan      | Um caminho de loader expôs objetos.                     | Submit, shader, rendering ou apresentação.      |
| Consulta de capacidade | A implementação reportou propriedade ou extensão.       | Execução correta em todos os estados.           |
| Alocação e mapeamento  | Um caminho de alocação e visibilidade de CPU funcionou. | A GPU acessou o recurso.                        |
| Gravação de comandos   | O caminho aceitou a construção.                         | Submit, execução ou conclusão.                  |
| Submit                 | A API aceitou uma requisição.                           | O firmware executou o trabalho.                 |
| Fence                  | O objeto de sincronização alcançou um estado.           | Que aquele trabalho específico causou o estado. |
| Readback esperado      | Uma operação fim a fim específica passou.               | Suporte geral a formatos, filas ou workloads.   |

## Referências

# Capítulo incorporado: 01-hardware-overview.md

## Hardware Overview

**Status:** confirmed for the supplied SM-S721B capture, with revision-specific details kept separate.

The current laboratory target is a Samsung SM-S721B ( `r12s` ) using platform `erd9945` and hardware `s5e9945` . The SGPU render node is `/dev/dri/renderD128` , attached to the platform device `/sgpu@22200000` and the kernel driver `sgpu` . The display path is separate: `/dev/dri/renderD129` is attached to `exynos-drm` .

| Field             | Observed value                          | Evidence label             |
|-------------------|-----------------------------------------|----------------------------|
| Model             | <code>SM-S721B</code>                   | Confirmed from device log  |
| Device codename   | <code>r12s</code>                       | Confirmed from device log  |
| Platform          | <code>erd9945</code>                    | Confirmed from device log  |
| Hardware property | <code>s5e9945</code>                    | Confirmed from device log  |
| GPU family        | <code>147 (M6FX)</code>                 | Confirmed from SGPU probe  |
| Device ID         | <code>0x000073a0</code>                 | Confirmed from SGPU probe  |
| Chip revision     | <code>0x02600200</code>                 | Confirmed from SGPU probe  |
| SGPU node         | <code>/dev/dri/renderD128</code>        | Confirmed from device log  |
| SGPU compatible   | <code>samsung-sgpu,samsung-sgpu</code>  | Confirmed from uevent/DT   |
| GFX IP            | one instance, version <code>10.0</code> | Confirmed for this capture |
| COMPUTE IP        | one instance, version <code>10.0</code> | Confirmed for this capture |
| DMA IP            | zero instances reported                 | Confirmed for this capture |
| Wave front size   | <code>32</code>                         | Confirmed for this capture |
| Active CUs        | <code>12</code>                         | Confirmed for this capture |

The device log also reports `ro.hardware.vulkan=samsung` and `ro.hwui.use_vulkan=true` . These properties establish the intended Android graphics configuration, not that a Termux process can load the Samsung ICD.

### Revision boundary

The source release includes an `s5e9945-sgpu_evt0.dtsi` file with `chip_revision = <0x02600100>` , while the observed device reports `0x02600200` . This difference is a reason to keep EVT0 source values separate from the observed sample until the exact build and Device Tree composition are correlated.

### What this does not prove

The hardware summary does not prove shader ISA compatibility with AMDGPU, implementação Vulkan de referência, or driver Vulkan móvel de referência. The AMDGPU-like family and identifiers appear in the kernel/UAPI layer, but user-space compiler and driver compatibility remain open questions.

# References

---

## Capítulo incorporado: 02-chip-revisions.md

### Chip Revisions

**Status:** planned investigation; no result is claimed until an evidence record is linked.

#### Purpose

Separate revision-specific observations and prevent facts from one SM-S721B build from being generalized to other Xclipse generations.

#### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

#### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

#### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

#### Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

# References

---

# Capítulo incorporado: 03-device-tree-and-platform.md

## Device Tree and Platform

**Status:** source-backed and cross-checked against the supplied device capture.

The Samsung source release defines the SGPU node for s5e9945 in `kernel/arch/arm64/boot/dts/exynos/s5e9945-sgpu_common.dtsi`. The node is `sgpu@22200000` with compatible string `samsung-sgpu,samsung-sgpu`. The observed uevent exposes the same compatible string and full name `/sgpu@22200000`.

| Property           | Source-backed value                                                                                                          | Meaning or follow-up                                                           |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Node               | <code>sgpu@22200000</code>                                                                                                   | Platform GPU node and address base.                                            |
| Compatible         | <code>samsung-sgpu,samsung-sgpu</code>                                                                                       | Matches <code>sgpu_kms_driver</code> in <code>amdgpu_drv.c</code> .            |
| Register regions   | <code>gpu</code> , <code>doorbell</code> , <code>debug</code> , <code>pwrctl</code> , <code>sysreg</code> , <code>htu</code> | Six MMIO regions are declared; access must remain source- and safety-reviewed. |
| Interrupts         | <code>SGPU</code> , <code>GPU-AFM</code>                                                                                     | GPU and adaptive-frequency-management interrupt sources.                       |
| Chip flag          | <code>CHIP_VANGOGH_LITE</code>                                                                                               | The driver maps this flag to device ID <code>0x73A0</code> .                   |
| GL2 ACEM instances | <code>4</code>                                                                                                               | Consumed during driver initialization.                                         |
| Power              | <code>pd_g3dcore</code>                                                                                                      | GPU power-domain dependency.                                                   |
| Coherency          | <code>dma-coherent</code>                                                                                                    | Source declaration; memory behavior still requires runtime validation.         |
| AFM                | PMIC source <code>2</code> , offset <code>0x20</code>                                                                        | Power safety and frequency limiting path.                                      |

`amdgpu_drv.c` reads `chip_revision` from the Device Tree unless a force parameter overrides it. It also reads the GL2 ACEM instance count and optionally the DVFS calibration ID. The Samsung driver registers a platform driver named `sgpu`, not only a conventional PCI driver.

### Revision-specific data

The EVT0 include adds `chip_revision = <0x02600100>` and DVFS/IFPO parameters. The observed sample reports `0x02600200`. This repository therefore treats Device Tree values as revision-scoped evidence, not universal Xclipse constants.

### Platform-to-GPU path

The observed path is: `/sys/firmware/devicetree/base/sgpu@22200000` → platform device `22200000.sgpu` → driver `/sys/bus/platform/drivers/sgpu` → DRM nodes `card0` and `renderD128`. The display controller is a separate `exynos-drm` path on `card1` and `renderD129`.

## References

### coleta de reprodução: correlação inicial com os nós DRM

O probe direto `X940-001c` observou `card0 / renderD128` reportando `amdgpu` e `card1 / renderD129` reportando `exynos-drmdpu`. Essa é uma identificação da camada DRM retornada pelo ioctl. A correlação física ainda deve seguir por major/minor, sysfs, driver, platform device e `sgpu@22200000`.

## Capítulo incorporado: 04-memory-model.md

### Memory Model

**Status:** first reversible memory path confirmed; full residency, cache, and synchronization model pending.

The SGPU probe successfully executed a bounded memory sequence on `/dev/dri/renderD128`: it requested a 64 KiB GEM object in the GTT domain with 4 KiB alignment, obtained a handle, queried a mmap offset, mapped it into the process, wrote `0xAB` across the allocation, unmapped the CPU view, mapped the object into GPU virtual address `0x80000000`, unmapped that VA range, and closed the GEM handle.

| Operation                                                  | Observed result                             | Interpretation                                            |
|------------------------------------------------------------|---------------------------------------------|-----------------------------------------------------------|
| <code>DRM_IOCTL_AMDGPU_GEM_CREATE</code>                   | <code>ok handle=1</code>                    | A GTT-domain GEM allocation path worked for this request. |
| <code>DRM_IOCTL_AMDGPU_GEM_MMAP</code> + <code>mmap</code> | <code>mmap+touch</code><br><code>ok</code>  | CPU mapping and access worked for this buffer.            |
| VA map                                                     | <code>ok</code><br><code>@0x80000000</code> | A VA mapping path accepted the requested range and flags. |
| VA unmap                                                   | <code>ok</code>                             | The mapping was removed in the same probe.                |
| GEM close                                                  | <code>ok</code>                             | The probe cleaned up the handle.                          |
| GPU page faults                                            | <code>faults=0</code>                       | No page faults were reported at the query point.          |

The source UAPI defines AMDGPU-derived domains and flags, including GTT, VRAM, CPU access, uncached, encrypted, and explicit synchronization flags. It defines `drm_amdgpu_gem_va` with map, unmap, clear, and replace operations, plus readable, writable, executable, and memory-type flags.

### Important limits

The successful CPU map does not prove that a GPU instruction read or wrote the buffer. The VA map does not prove that a command processor consumed the mapping. No command stream was submitted by `sgpu_raw_probe.c`, and no GPU readback was performed. The next memory phase must therefore study cache visibility, DMA-BUF ownership, residency, page tables, faults, and synchronization before any custom submission.

The device capture reports `dma-coherent` in the Device Tree and `ids_flags=0x4` in the device information. These are source/driver signals that require careful interpretation; they are not a complete cache-coherency specification.

References

---

Capítulo incorporado: 05-mmuvirtual-addressing.md

MMU and Virtual Addressing

**Status:** basic VA mapping confirmed; page-table format and fault recovery remain open.

The observed SGPU device reports a virtual-address offset of 0x8000000 , maximum address 0x800000000000 , alignment 0x1000 , and GART page size 0x1000 . The raw probe uses the same offset for a temporary 64 KiB mapping with readable and writable flags, then removes it.

| Field                | Observed value      | Confidence              |
|----------------------|---------------------|-------------------------|
| VA offset            | 0x8000000           | Confirmed for capture   |
| VA maximum           | 0x800000000000      | Confirmed for capture   |
| VA alignment         | 0x1000              | Confirmed for capture   |
| GART page size       | 0x1000              | Confirmed for capture   |
| Probe flags          | readable + writable | Confirmed from source   |
| Page faults at query | 0                   | Confirmed at probe time |

The UAPI header exposes AMDGPU\_VA\_OP\_MAP , AMDGPU\_VA\_OP\_UNMAP , AMDGPU\_VM\_PAGE\_READABLE , AMDGPU\_VM\_PAGE\_WRITEABLE , and AMDGPU\_VM\_PAGE\_EXECUTABLE . It also defines memory-type flags and delayed page-table update semantics. These definitions are an interface surface, not a complete hardware page-table specification.

What remains unknown

The project still needs the page-table hierarchy, PTE encoding, invalidation timing, fault reporting, residency rules, protected-memory behavior, and interaction between TTM/GART/IOMMU and the SGPU firmware. The probe’s successful map/unmap sequence is a prerequisite result, not proof that executable GPU virtual memory is ready for custom work.

Safety boundary

Do not test arbitrary GPU virtual addresses, executable mappings, or opaque command buffers until reset and recovery are understood. Keep mappings bounded and cleanly released.

References

---

# Capítulo incorporado: 06-registers.md

## Registers

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Maintain a source-backed and capture-backed register map with access width, reset value, side effects, and safety notes.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

### Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

### References

---

# Capítulo incorporado: 07-command-processor.md

## Command Processor

**Status:** UAPI and source paths identified; no custom command submission has been demonstrated.

### Purpose

The supplied `sgpu_raw_probe.c` intentionally stops before command submission. Its source comment states `Submits NOTHING to the GPU`; the observed output ends after GEM close with `DONE fails=2`. Therefore the package does not contain a packet capture or a proven command-processor execution trace.

The Samsung UAPI nevertheless exposes an AMDGPU-derived command-stream interface. `drm_amdgpu_cs_in` carries a context ID, BO-list handle, chunk count, flags, and a pointer to chunks. `drm_amdgpu_cs_chunk` identifies an IB, fence, dependency, sync object, BO-handle, or timeline operation. `drm_amdgpu_cs_chunk_ib` carries a virtual address, byte size, IP type, IP instance, ring, and flags such as secure, preempt, cache synchronization, performance counter, and SQ thread trace.

| Evidence                                                              | What it establishes                                                                                     | What it does not establish                                  |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| UAPI <code>DRM_IOCTL_AMDGPU_CS</code> definition                      | A command submission contract exists in the source interface.                                           | That the SGPU accepts every upstream AMDGPU packet or flag. |
| UAPI IB/chunk structures                                              | The shape of a possible submission path.                                                                | The packet encoding placed inside an IB.                    |
| Observed GFX/COMPUTE ring masks <code>0xf</code> and <code>0x7</code> | Ring availability was reported by <code>HW_IP_INFO</code> .                                             | Which ring is safe or correct for custom work.              |
| <code>sgpu_raw_probe</code>                                           | Memory and VA prerequisites worked.                                                                     | Any GPU command executed.                                   |
| <code>libdrm_sgpu.so</code> symbols                                   | Wrappers for <code>amdgpu_cs_submit</code> and <code>sgpu_cs_submit</code> exist in the vendor library. | ABI compatibility or safe use outside its intended runtime. |

### Required next evidence

Before constructing an IB, document context creation, BO lists, firmware mediation, synchronization, ring selection, reset behavior, and a bounded recovery path. A real execution test must submit known work and validate an independent result. Opaque command-buffer replay is out of scope until provenance and safety are understood.

### References

#### Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de `BO_LIST` e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

# Capítulo incorporado: 08-command-packets.md

## Command Packets

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Define packet hypotheses only from controlled captures and state which fields remain unknown or unsafe to replay.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

### Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

### References

#### Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em

A1.5\_REAL\_CS\_20260907\_161914 , um DRM\_IOCTL\_AMDGPU\_CS foi aceito ( ioctl\_ret=0 , CS\_IOCTL=ACCEPTED ) para um chunk IB ( chunk\_id=0x1 ) com VA 0x4000000000 e ib\_bytes=4 .

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com EINVAL ou EFAULT / Bad address . Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e dmesg permanecem fora do repositório.

## Capítulo incorporado: 09-queues-and-scheduling.md

### Queues and Scheduling

**Status:** IP inventory confirmed; queue lifecycle and safe submission remain unvalidated.

The raw SGPU probe reports one GFX IP instance, version 10.0 , ring mask 0xf , IB alignment 32/32 ; one COMPUTE IP instance, version 10.0 , ring mask 0x7 , IB alignment 32/32 ; and zero DMA instances. These values describe what the kernel query returned for this sample. They do not identify a safe ring for a new user-space submission.

The source release includes amdgpu\_ring , amdgpu\_job , amdgpu\_sched , amdgpu\_fence , amdgpu\_sync , context, BO-list, and reset-query code in the SGPU module. The UAPI includes context priorities, reset status, dependencies, fences, sync objects, timeline wait/signal chunks, and scheduler priority operations.

| Question                                      | Current answer                                                          | Evidence label            |
|-----------------------------------------------|-------------------------------------------------------------------------|---------------------------|
| How many GFX/COMPUTE instances were reported? | One each.                                                               | Confirmed for capture     |
| Which ring masks were reported?               | GFX 0xf ; COMPUTE 0x7 .                                                 | Confirmed for capture     |
| Was work submitted by the supplied probe?     | No.                                                                     | Confirmed from source     |
| Is preemption supported?                      | The UAPI exposes preempt-related flags; runtime behavior is not proven. | Probable/source-backed    |
| Is reset status query available?              | Context and reset-query structures exist in UAPI and vendor symbols.    | Confirmed by source       |
| Is a queue completion path proven?            | No.                                                                     | Confirmed negative status |

### Next experiment boundary

The next queue experiment must begin with context and synchronization documentation, use a fresh bounded BO, submit only a known minimal IB, wait with a documented fence or sync object, and have a recovery procedure. It must never infer execution from a successful ioctl alone.

# References

## Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

# Capítulo incorporado: 10-synchronization.md

## Synchronization

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Document fences, sync objects, semaphores, cache visibility, timeline semantics, and completion evidence.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

## Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

## References

### Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

## Capítulo incorporado: 11-firmware.md

## Firmware

**Status:** version metadata confirmed; load order, contents, and redistribution status remain open.

The device exposes SGPU firmware metadata through sysfs and the SGPU information query. The supplied capture reports SGPU firmware `2.23.0` and RTL change-list number `0x0004ea15`. Component queries returned ME `0x00000005`, MEC `0x00000004`, PFP `0x00000007`, and RLC `0x00000001`. CE, MC, SDMA, SDMA2, SMC, and the RLC restore-list components reported zero in this capture.

| Component      | Observed value          | Interpretation                                                    |
|----------------|-------------------------|-------------------------------------------------------------------|
| SGPU           | <code>2.23.0</code>     | Vendor-facing SGPU version string.                                |
| SGPU RTL CL    | <code>0x0004ea15</code> | Runtime metadata associated with this build.                      |
| ME             | <code>0x5</code>        | Firmware query result.                                            |
| MEC            | <code>0x4</code>        | Firmware query result.                                            |
| PFP            | <code>0x7</code>        | Firmware query result.                                            |
| RLC            | <code>0x1</code>        | Firmware query result.                                            |
| CE/MC/SDMA/SMC | <code>0x0</code>        | Zero returned; meaning must be verified against driver semantics. |

The source release contains `amdgpu_ucode.c`, `amdgpu_atomfirmware.c`, and signed/unified firmware header material for multiple revisions. The Kconfig can enable built-in firmware, while the default shown for that option is disabled. These source paths document the integration mechanism; they do not grant permission to redistribute runtime blobs or claim that every source variant corresponds to the tested handset.

## Open questions

The project still needs the exact firmware file names loaded on the SM-S721B, load order, signature checks, firmware-to-revision mapping, command-processor responsibilities, and reset behavior. Keep firmware binaries outside Git until provenance and license review are complete.

## References

---

# Capítulo incorporado: 12-reset-and-recovery.md

## Reset and Recovery

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Define safe recovery boundaries, reset causes, GPU hang handling, and experiment rollback procedures.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

### Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

## References

### Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

## Capítulo incorporado: 13-power-clocks-and-thermal.md

### Power, Clocks, and Thermal

**Status:** planned investigation; no result is claimed until an evidence record is linked.

#### Purpose

Record power domains, clock rates, OPPs, thermal constraints, and their effects on reproducibility.

#### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

#### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

#### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

## Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

## References

# Capítulo incorporado: 14-shader-isa.md

## Shader ISA

**Status:** strong static evidence of shader/compiler/encoding infrastructure; no native Xclipse 940 ISA stream has been captured.

The new Etapa 5 analysis inspected the stripped AArch64 `vulkan.samsung.so` and found strings and code references associated with SPIR-V, shader compilation, opcode translation, hardware opcode selection, instruction emission, and encoding. Relevant names include `SCEmitterGFX103.cpp`, `SCAsmEncoder.cpp`, `SCAsmEncoder.hpp`, `GetOpcode`, `gen_opcode`, `XLateOpcode`, `GetHwOpcode`, `EncodeDPP`, `EncodeSDWA`, `EncodeWaitDepctr`, `EncodeImmediateBuffer`, `EncodeMaccDelay`, `SEmitVOp1`, `SEmitScratch`, `SEmitFlat`, `VOPC/VOP1/VOP2/VOP3/VOP3P`, `DPP8`, and `DPP16`.

The ELF is AArch64, stripped, and carries BuildID `7b6134ba45f561f28b006e07ef075b4d4c429bcd`. The `.text` virtual address was correctly distinguished from the file offset: the disassembly began at `0x143fb30`. This produced real AArch64 instructions, but the first inspected region was not proven to be shader compilation code.

| Evidence                                                                                               | Defensible interpretation                                                                           |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| SPIR-V, shader, compiler, pipeline strings                                                             | The vendor binary contains shader-related components or diagnostics.                                |
| <code>GetOpcode</code> , <code>XLateOpcode</code> , <code>GetHwOpcode</code> , <code>gen_opcode</code> | Strong static evidence of opcode-handling concepts; stripped strings are not symbols by themselves. |
| <code>SCAsmEncoder*</code> , <code>Encode*</code> , VOP/DPP families                                   | Strong evidence of an internal assembly/encoding vocabulary.                                        |
| References ending at <code>__android_log_print</code>                                                  | Some occurrences are logging/debug paths, not the implementation named by the string.               |
| <code>gfx10_4_GEN</code> and <code>MGFX*_GEN</code>                                                    | Do not prove that Xclipse 940 is GFX10.4 or that a specific backend is used at runtime.             |
| Real AArch64 disassembly                                                                               | Confirms native code inspection, not the ISA of the generated shader.                               |

The current classification is therefore **static compiler/encoding infrastructure strongly supported**, not “Xclipse ISA decoded.”

The package still lacks the chain `controlled SPIR-V → Samsung-generated/consumed binary → runtime execution → native instruction correlation`.

## Required next evidence

Obtain a controlled shader through the production Samsung path, capture binary or executable metadata, and correlate at least one instruction or encoding field with a known operation and a runtime result. Keep textual diagnostics, compiler labels, and log strings separate from actual ISA bytes.

# References

---

## Capítulo incorporado: 15-wave-execution.md

### Wave Execution

**Status:** planned investigation; no result is claimed until an evidence record is linked.

#### Purpose

Investigate execution width, scheduling, occupancy, barriers, and per-wave state only after execution evidence exists.

#### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

#### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

#### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

#### Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

# References

---

# Capítulo incorporado: 16-register-allocation.md

## Register Allocation

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Separate compiler register naming from hardware allocation, spills, liveness, and observable binary effects.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

### Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

### References

---

# Capítulo incorporado: 17-textures-and-images.md

## Textures and Images

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Document formats, tiling, swizzles, layouts, compression, views, and synchronization requirements.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

### Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

### References

---

# Capítulo incorporado: 18-rendering-pipeline.md

## Rendering Pipeline

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Map vertex, primitive, raster, fragment, attachment, blend, and presentation behavior through executable tests.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

### Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

### Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

### References

---

# Capítulo incorporado: 19-compute-pipeline.md

## Compute Pipeline

**Status:** a real Samsung compute path is identified; end-to-end project-controlled dispatch and readback are not demonstrated.

The new Etapa 4 evidence corrected the process name and found the supported Photo Remaster path.

`com.samsung.android.photoremasterservice:photoremasterservice` loaded `vulkan.samsung.so`, `libdrm_sgpu.so`, `libOpenCL.so`, and `libSGPUOpenCL.so`, with FDs for `/dev/dri/renderD128` and `/dev/dri/card0`.

The OpenCL library exposes context and queue creation, buffers/images, program and kernel creation, `clEnqueueNDRangeKernel`, `clEnqueueTask`, `clFlush`, `clFinish`, events, barriers, and `clEnqueueReadBuffer`.

During an observed Photo Remaster operation, the SGPU runtime was reported active, frequency observations changed from 252000 to 500000, and the service continued to hold SGPU DRM FDs. This establishes a strong correlation between a real vendor workload and SGPU activity. It does not identify the exact kernel, prove a successful submit, or prove a validated buffer transition.

| Required step              | New evidence                                                                                          | Status                             |
|----------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------|
| Reach the production stack | Photo Remaster service loads Vulkan, libdrm, and SGPU OpenCL.                                         | Confirmed path                     |
| Allocate a resource        | Previous 64 KiB GEM/BO probe exists.                                                                  | Partial; CPU/GTT path only         |
| Record a dispatch          | OpenCL symbols include <code>clEnqueueNDRangeKernel</code> ; no project-controlled call trace exists. | Capability present, execution open |
| Submit                     | <code>sgpu_cs_submit</code> and OpenCL execution APIs are present; no successful project trace.       | Open                               |
| Synchronize                | OpenCL finish/events/barriers and SGPU sync APIs are available.                                       | Interface mapped, event proof open |
| Validate readback          | <code>clEnqueueReadBuffer</code> exists; no known input/output pair was captured.                     | Open                               |

The minimum compute claim remains:

`B0 before` → `real submit` → `GPU execution` → `fence/event` → `B0 after/readback` → `recovery`

A line such as `compute pipeline ok`, an exported symbol, active frequency, or a vendor workload correlation is not sufficient. The next safe milestone is to observe a controlled Photo Remaster operation or another supported vendor workload with known input and output, before attempting any direct submission experiment.

## References

### Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de `BO_LIST` e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este

resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

## Capítulo incorporado: 20-vulkan-mapping.md

### Vulkan Mapping

**Status:** loader observation and pipeline metadata probe documented; independent Vulkan execution not demonstrated.

The supplied Vulkan probe uses no SDK headers. It loads `libvulkan.so`, creates a `VkInstance`, enumerates physical devices, selects vendor `0x144d` if visible, searches for `VK_KHR_pipeline_executable_properties`, creates a minimal device and compute pipeline from embedded SPIR-V, and queries executable properties, statistics, and internal representations.

The probe does **not** retrieve a device queue, record a command buffer, call `vkCmdDispatch` or `vkCmdDraw`, submit work, wait for a fence, or read back a result. Consequently, `compute pipeline ok` means that pipeline creation returned success; it does not mean that a compute workload executed.

| Vulkan layer                  | Supplied evidence                                                 | Current interpretation                                    |
|-------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------|
| Loader open                   | <code>libvulkan opened</code>                                     | The process opened the loader visible to its namespace.   |
| Instance                      | <code>instance ok</code>                                          | Instance creation worked for that loader.                 |
| Physical devices              | One device: <code>llvmpipe</code> , vendor <code>0x10005</code> . | Software Vulkan was visible.                              |
| Samsung selection             | <code>0x144d</code> not visible.                                  | The observed process did not reach the Samsung ICD.       |
| Pipeline executable extension | Not reached on the Samsung path.                                  | Availability remains unproven through the correct bridge. |
| Queue/dispatch/readback       | Not present in source.                                            | No Vulkan compute execution claim is allowed.             |

### Mapping priorities

Once the Android loader path is understood, the first executable Vulkan milestone should be device selection, queue retrieval, a bounded buffer, a minimal compute dispatch, synchronization, and validated readback. Images, textures, rendering, presentation, descriptors, pipeline cache, and layers should follow from evidence rather than from the existence of API entry points.

### References

#### Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este

resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

### coleta de reprodução: limite da identificação DRM

O nome `amdgpu` observado por `DRM_IOCTL_VERSION` em `renderD128` é uma pista de compatibilidade estrutural, não uma confirmação de ABI AMDGPU upstream, ISA AMD ou implementação Vulkan de referência. O mapeamento Vulkan deve continuar separado da identificação do driver DRM.

## Capítulo incorporado: 21-android-loader.md

### Android Loader

**Status:** the production Samsung loading path is now observed; detailed Vulkan enumeration from that process remains open.

The supplied Etapa 1 evidence captured `SurfaceFlinger` with PID 995 while it was alive. The process mapped both `/system/lib64/libvulkan.so` and `/vendor/lib64/hw/vulkan.samsung.so`, and held file descriptors pointing to `/dev/dri/renderD128`. Its mount namespace was `mnt:[4026535441]`; the observed Termux process used `mnt:[4026535972]`. This explains why Termux exposed only `llvmpipe` without implying that the Samsung ICD is absent.

The first attempt to inspect PID 995 was invalid because the process had already exited. `/proc/995/status`, `/proc/995/maps`, and its namespace directory were missing in that snapshot. A later, time-consistent capture supplied the valid maps and DRM file descriptors. The project must retain this distinction.

| Component                  | Observed state                                                                                     | Confidence                            |
|----------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------|
| System loader              | <code>/system/lib64/libvulkan.so</code> mapped in SurfaceFlinger.                                  | Confirmed in a valid process snapshot |
| Vendor ICD                 | <code>/vendor/lib64/hw/vulkan.samsung.so</code> mapped in SurfaceFlinger.                          | Confirmed in a valid process snapshot |
| Production DRM path        | SurfaceFlinger FDs point to <code>/dev/dri/renderD128</code> .                                     | Confirmed in a valid process snapshot |
| Termux loader result       | <code>llvmpipe</code> , vendor <code>0x10005</code> .                                              | Confirmed for Termux only             |
| Samsung Vulkan enumeration | A complete <code>vkEnumeratePhysicalDevices</code> capture from SurfaceFlinger is not yet present. | Open                                  |
| Direct ICD loading         | Termux linker namespace rejected the vendor path and the attempt ended in a crash.                 | Confirmed for that approach           |

The defensible conclusion is now stronger than “vendor library exists”: a production Android process maps the Samsung ICD and opens the SGPU render node. It is still not evidence that an external Mesa loader can call the vendor library, nor that Termux can join the production namespace. Root and temporary SELinux changes do not substitute for namespace membership.

### Next safe observation

Capture a supported process while it enumerates Vulkan devices and records vendor/device IDs, extensions, and the relation to the graphics service. Do not replace `/system` or `/vendor` libraries and do not treat a dead-PID snapshot as evidence.

## References

---

### coleta de reprodução: condição de reprodução no Android

O workspace coleta de reprodução registrou que o armazenamento compartilhado pode ser montado com `noexec`. Probes devem ser compilados e executados em uma área de trabalho executável e somente depois arquivados em `armazenamento compartilhado`.

---

## Capítulo incorporado: 22-linker-namespaces.md

### Linker Namespaces

**Status:** one namespace boundary is confirmed; namespace identity and the production bridge remain to be mapped.

The direct ICD experiment attempted to open `/vendor/lib64/hw/vulkan.samsung.so` from the Termux process and received a linker warning that the library was not accessible for the namespace, followed by a segmentation fault. Opening `/system/lib64/libvulkan.so` did work, but it exposed only llvmpipe. A root shell and temporary SELinux `Permissive` state did not alter this result.

| Observation                                          | Evidence label                | Limit                                                          |
|------------------------------------------------------|-------------------------------|----------------------------------------------------------------|
| Vendor path is inaccessible to the Termux namespace. | Confirmed for this approach   | Does not name the namespace or prove every process is blocked. |
| Root did not change loader visibility.               | Confirmed for this experiment | Root does not imply linker-namespace membership.               |
| System loader is callable.                           | Confirmed for this process    | It may select different ICDs in another Android process.       |

### Next safe work

Identify the production graphics process and its linker configuration using read-only inspection. Do not modify namespace configuration, `/system`, `/vendor`, manifests, or persistent properties. Do not use a segmentation fault as evidence of ICD incompatibility; it proves only that this loading approach is invalid in this namespace.

## References

---

## Capítulo incorporado: 23-icd-and-manifests.md

### ICD and Manifests

**Status:** the vendor ICD is confirmed in the production SurfaceFlinger process; standalone external-loader use and full Vulkan enumeration remain open.

The device report identifies `/vendor/lib64/hw/vulkan.samsung.so` at 44,423,944 bytes and `/system/lib64/libvulkan.so` at 240,208 bytes. The new Etapa 1 capture shows both libraries mapped inside `SurfaceFlinger` PID 995 while valid, together with FDs pointing to `/dev/dri/renderD128`. This is stronger than a file-existence check and establishes a real production loading path.

The vendor ELF is AArch64, stripped, Android NDK r26, with BuildID `7b6134ba45f561f28b006e07ef075b4d4c429bcd`. Its dependencies include Android/vendor libraries such as `libhardware.so`, `libsync.so`, `libsbwcheelper.so`, and `android.hardware.graphics.mapper@4.0-impl-sgr.so`. These dependencies explain why copying the library into Termux or treating it as a standalone Mesa ICD is not a valid equivalence.

| Artifact                                                   | Meaning                                                               | Current status                                                   |
|------------------------------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------|
| <code>vulkan.samsung.so</code>                             | Vendor Vulkan implementation mapped by SurfaceFlinger.                | Production presence confirmed; external standalone use unproven. |
| <code>libvulkan.so</code>                                  | Android/system loader mapped by SurfaceFlinger and visible to Termux. | Present; Termux selected llvmpipe.                               |
| <code>SurfaceFlinger</code><br><code>renderD128</code> FDs | Production process reaches SGPU DRM.                                  | Confirmed in valid snapshot.                                     |
| Vulkan JSON                                                | Standard external-loader manifest.                                    | Not found in the searched paths.                                 |
| OpenCL <code>samsung.icd</code>                            | OpenCL manifest, not Vulkan JSON.                                     | Must not be classified as Vulkan manifest.                       |

The difference between the `SurfaceFlinger` and Termux mount namespaces is material. A successful production mapping does not grant Termux access, and exported Vulkan entry points do not prove that an external Mesa loader can call the library.

## References

# Capítulo incorporado: 24-layers.md

## Diagnostic Layers

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Build tracing and validation layers only after a genuine target ICD path is available; layers cannot replace an ICD.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

## Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

## Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

## Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

## References

---

# Capítulo incorporado: 25-kernel-drm-uapi.md

## Kernel DRM and UAPI

**Status:** SGPU node, UAPI, platform path, firmware, vendor libdrm and submission interfaces are mapped; a custom submission remains unproven.

The observed DRM layout contains SGPU `card0 / renderD128` and display `card1 / renderD129`. The SGPU path is `/sys/devices/platform/22200000.sgpu`, bound to `/sys/bus/platform/drivers/sgpu`, with compatible `samsung-sgpu,samsung-sgpu` at `/sgpu@22200000`.

The source release contains `kernel/include/uapi/drm/sgpu_drm.h` and the SGPU implementation tree. The new package additionally inventories `/vendor/lib64/libdrm_sgpu.so` and symbols for BO allocation/import/export, BO lists, VA operations, context creation, submission, raw submission, reset queries, page-fault queries, fences, semaphores, and sync objects.

| Layer              | Confirmed                                                                                | Not yet confirmed                                                |
|--------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Node discovery     | SGPU render node, platform binding, <code>card0</code> , <code>renderD128</code> .       | Full ABI behavior across builds.                                 |
| Device identity    | SGPU firmware 2.23.0, RTL <code>0x0004ea15</code> , devfreq and platform identity.       | Meaning of every revision-specific field.                        |
| Memory             | Previous 64 KiB GTT BO, CPU map/touch, VA map/unmap.                                     | GPU access, residency, cache protocol, page-fault recovery.      |
| Vendor libdrm      | <code>libdrm_sgpu.so</code> and relevant exported/string-visible interfaces are present. | A trace proving which interfaces were called and succeeded.      |
| Command submission | UAPI and context/submit/sync interfaces are documented.                                  | Safe custom context, IB, submit, fence, execution, and readback. |

An API name such as `sgpu_cs_submit` is evidence of an available interface, not of a successful submission. The next experiment must correlate a real process, its DRM FD, BO/VA state, synchronization object, and validated output.

## References

### Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

### coleta de reprodução: probe direto de DRM

O experimento `X940-001c` chama `DRM_IOCTL_VERSION` diretamente, sem depender de `libdrm`, e preserva fonte, binário, ambiente, stdout, stderr e hashes. O resultado `name=[amdgpu]` em `renderD128` deve ser tratado como identificação da camada DRM reportada, não como prova de compatibilidade com AMDGPU upstream ou implementação Vulkan de referência.

## Capítulo incorporado: 26-compiler-backend.md

### Compiler Backend

**Status:** no open Xclipse compiler backend is present in the supplied project evidence.

The package contains a Vulkan probe with embedded SPIR-V and calls to pipeline executable property interfaces. It does not contain a NIR/LLVM lowering pass, instruction selector, register allocator, code emitter, assembler, or disassembler for Xclipse. The Samsung source release contains kernel-side GPU support and register/firmware material, which is useful reference evidence but is not a user-space compiler backend.

| Needed component                  | Current state                       |
|-----------------------------------|-------------------------------------|
| Shader IR input                   | Minimal SPIR-V exists in the probe. |
| Xclipse instruction specification | Not established.                    |
| Instruction selection             | Not present.                        |
| Register allocation               | Not present.                        |
| Code emission                     | Not present.                        |
| Binary validation                 | Not present.                        |
| Execution test vectors            | Not present.                        |

The compiler phase must begin only after the Samsung device produces controlled shader binaries or other trustworthy representations. Source reuse also requires file-level license review.

## References

# Capítulo incorporado: 27-known-errata.md

## Known Errata

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Record confirmed failures, environment-specific limitations, unsafe approaches, and unresolved discrepancies.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

## Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

## Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

## References

---

# Capítulo incorporado: 28-security-boundaries.md

## Security Boundaries

**Status:** planned investigation; no result is claimed until an evidence record is linked.

### Purpose

Document root, SELinux, linker, firmware, memory, capture, and publication boundaries without weakening device security.

### Current boundary

The project plan identifies this area as necessary for an independent Xclipse driver, but the supplied plan is not itself proof that the area has been implemented or experimentally validated. This chapter must distinguish source-backed facts, direct observations, probable interpretations, hypotheses, and discarded approaches.

### Evidence table

| Question            | Evidence required                                            | Current state                 | Confidence                                |
|---------------------|--------------------------------------------------------------|-------------------------------|-------------------------------------------|
| What is known?      | Raw artifact, source path, or reproducible output.           | Initial plan only.            | Hypothesis until an artifact is attached. |
| What is executable? | A test that reaches the target layer and validates a result. | Not demonstrated by the plan. | Not established.                          |
| What can be reused? | License and provenance review.                               | Pending archive inventory.    | Not established.                          |

## Method

Start with read-only observations and source inventory. Preserve raw outputs without cosmetic edits. Add an interpretation report beside each raw artifact. If a harness initializes a test but does not submit and validate work on the target GPU, record it as an initializer or probe rather than an execution test.

## Open questions

The chapter remains open until the relevant source paths, device revision, firmware context, safety boundary, and reproducible validation procedure are documented.

## References

---

# Capítulo incorporado: 29-validation-results.md

## Validation Results

## Validation Results

This document is a results index, not a claim that all planned milestones have passed. It records the strongest evidence currently extracted from the supplied package.

| Milestone         | Current state                                                                                                   | Evidence status                                   |
|-------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| Platform identity | SM-S721B ( r12s ), erd9945 , s5e9945 ; SGPU node at /dev/dri/renderD128 .                                       | Confirmed from device logs.                       |
| Kernel/UAPI       | sgpu_drm.h , SGPU driver tree, s5e9945 Device Tree, IOMMU and DMA-BUF paths inventoried.                        | Confirmed by source; runtime correlation pending. |
| Android loader    | System loader and vendor ICD paths are present; Termux sees only llvmpipe.                                      | Confirmed for observed process.                   |
| GPU enumeration   | DRM probe identifies MGFX family 147 , device 0x73a0 ; Vulkan vendor 0x144d is not visible in Termux namespace. | Confirmed for separate paths.                     |
| Memory/VM         | 64 KiB GTT BO, CPU mmap/touch, VA map/unmap and close succeeded.                                                | Confirmed for capture.                            |
| Queue inventory   | One GFX and one COMPUTE IP instance reported; ring masks 0xf and 0x7 .                                          | Confirmed for capture; no submit.                 |
| Firmware metadata | SGPU 2.23.0 , RTL 0x4ea15 , component versions recorded.                                                        | Confirmed for capture.                            |
| Compute execution | No queue submit, dispatch, synchronization, or validated readback exists in supplied probes.                    | Not demonstrated.                                 |
| ISA               | No instruction format decoded with confidence; pipeline IR path not reached on Samsung GPU.                     | Not demonstrated.                                 |
| Compiler          | No Xclipse compiler backend.                                                                                    | Not started.                                      |
| Vulkan            | Instance and pipeline metadata probe exists; no independent target-device execution path.                       | Partial observation only.                         |
| Layers            | No diagnostic layer proven on Samsung ICD.                                                                      | Not started.                                      |
| Driver            | No independent Vulkan driver feature demonstrated.                                                              | Not started.                                      |

## Interpretation rule

A row can move to “confirmed” only when the repository contains an evidence report that proves the exact milestone. The existence of planned directories, initializers, capability queries, pipeline creation, or buildable stubs does not change this table.

## References

### Addendum 2026-09-07 — xclipselogs

A nova rodada xclipselogs.zip acrescentou evidência de um cliente DRM próprio em /dev/dri/renderD128 . O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em A1.5\_REAL\_CS\_20260907\_161914 , um DRM\_IOCTL\_AMDGPU\_CS foi aceito ( ioctl\_ret=0 , CS\_IOCTL=ACCEPTED ) para um chunk IB ( chunk\_id=0x1 ) com VA 0x4000000000 e ib\_bytes=4 .

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com EINVAL ou EFAULT / Bad address . Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e dmesg permanecem fora do repositório.

## coleta de reprodução: nova evidência de reprodução

A documentação de reprodução registra experimentos completos e resultados negativos. X940-001b demonstrou que `Permission denied` pode ser causado por `noexec` no armazenamento compartilhado. X940-001c demonstrou a tabela de quatro nós DRM por ioctl direto. Esses resultados melhoram a reprodução, mas não constituem prova de execução GPU.

# Capítulo incorporado: 30-npu-enn-rootless.md

## NPU Exynos: NNAPI/ENN rootless

Data da atualização: 2026-09-18

Dispositivo de referência: Samsung SM-S721B / Exynos 2400

Classificação: execução NNAPI/ENN confirmada sem root para os grafos reproduzidos

### Conclusão

A rota pública NNAPI/ENN já foi demonstrada por um processo sem root. O processo obteve o serviço `android.hardware.neuralnetworks.IDevice/enn`, encontrou dois dispositivos NNAPI (`enn` e `nnapi-reference`), compilou e executou um grafo `SOFTMAX` no dispositivo `enn` e recebeu uma saída numérica correta.

A prova não demonstra que o serviço proprietário

`vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default` esteja aberto a aplicativos. As tentativas de descobrir esse serviço retornaram nulo tanto com UID comum quanto com root. Portanto, o caminho comprovado para um aplicativo é **NNAPI pública** → **HAL ENN** → **NPU**, e não a interface vendor proprietária direta.

### Prova principal sem root

O teste `item10_nnapi_hal_20260917_235059` obteve um Binder remoto para `android.hardware.neuralnetworks.IDevice/enn`. O `AIBinder_ping()` retornou `STATUS_OK`. O teste `item11_softmax_enn_20260918_000604` encontrou o dispositivo `enn` e reportou `SOFTMAX` como suportado. O teste `item12_exec_enn_20260918_001024` completou `ModelFinish`, compilação e `Execution_compute` sem root.

A saída do driver foi `[0.032059, 0.087144, 0.236883, 0.643914]`, igual ao resultado esperado, com diferença máxima `0.000000`. Essa é uma prova de execução funcional no dispositivo ENN por um processo comum. Ela não é apenas enumeração, compilação ou presença de biblioteca.

### O que mudou em relação à análise anterior

A análise anterior tratava `SOFTMAX` como rejeitado ou inviável com base no benchmark antigo. A nova coleta separa duas perguntas diferentes:

1. **O ENN consegue executar um SOFTMAX pequeno?** Sim, comprovado sem root.
2. **O ENN é rápido para um SOFTMAX maior?** Ainda não. O benchmark anterior mediu aproximadamente 1,0264 ms no ENN contra 0,0104 ms na CPU de referência para um vetor de 1024 elementos.

Assim, `SOFTMAX` deve ser removido da lista de operações universalmente rejeitadas. A classificação correta é: **suportado em pelo menos um grafo pequeno, porém com desempenho ruim no benchmark disponível**.

`BATCH_MATMUL` continua sem suporte no dispositivo testado. No teste corrigido, a operação foi adicionada ao modelo, mas `getSupportedOperationsForDevices` reportou que o `enn` não a suportava e a compilação terminou com status 4.

## Superfície do sistema confirmada

A coleta identificou:

- serviço `android.hardware.neuralnetworks.IDevice/enn` ;
- manifesto VINTF `android.hardware.neuralnetworks-service-enn.xml` ;
- binário `android.hardware.neuralnetworks-service-enn` ;
- runtime `libenn_wrapper.so` , `libenn_engine.so` , `libenn_model.so` e bibliotecas relacionadas;
- `libenn_public_api_cpp.so` e `libnpu_compiler.so` no firmware;
- modelos `.nnc` presentes em componentes do sistema;
- processo HAL no domínio SELinux `hal_neuralnetworks_service_enn_default` ;
- dispositivo `/dev/vertex10` , associado ao driver `exynos-npu` e ao nó `/npu_exynos` .

A presença desses elementos é evidência da arquitetura instalada. Ela não autoriza redistribuir bibliotecas, modelos, firmware ou código Samsung.

## Rota direta vendor: resultado atual

A interface `vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default` não foi descoberta pelo processo de teste. O resultado foi nulo em comparação entre UID comum e root. Isso pode significar que o serviço não está registrado naquele momento, que a interface é ativada apenas por consumidores específicos ou que há uma fronteira de namespace/política diferente. A coleta não permite afirmar que a interface não exista no sistema; permite afirmar que ela não foi uma rota funcional no teste realizado.

O endpoint `/dev/vertex10` foi aberto com root depois de desativar temporariamente o enforcing do SELinux, mas o UID comum recebeu `Permission denied` . Os ioctl's testados retornaram `EINVAL` ou `EFAULT` . Portanto, o endpoint não é atualmente uma API rootless comprovada e não deve ser usado diretamente por aplicativos comuns.

## Implicação para a pesquisa NPU

Uma futura biblioteca de pesquisa pode começar como uma camada de espaço de usuário baseada em NNAPI pública. A primeira implementação deve usar o dispositivo `enn` , consultar operações com `getSupportedOperationsForDevices` , compilar somente grafos aceitos e manter CPU como fallback.

O caminho de modelo deve ser particionado. `FULLY_CONNECTED INT8` já possui evidência de execução e checksum. `SOFTMAX` possui evidência de execução correta em grafo pequeno, mas precisa de otimização e medição. `BATCH_MATMUL` não pode ser enviado ao ENN no formato atual. Qualquer integração com outro aplicativo deve medir o custo real antes de escolher o backend.

## Nível de evidência

| Afirmação                                           | Classificação                                |
|-----------------------------------------------------|----------------------------------------------|
| Serviço público <code>IDevice/enn</code> existe     | Confirmada                                   |
| UID comum consegue obter Binder remoto do ENN       | Confirmada                                   |
| UID comum consegue compilar um grafo pequeno no ENN | Confirmada                                   |
| UID comum consegue executar SOFTMAX no ENN          | Confirmada                                   |
| SOFTMAX é rápido em workloads maiores               | Não demonstrada; benchmark anterior foi ruim |
| BATCH_MATMUL é aceito pelo ENN                      | Negativa para o formato testado              |
| Interface vendor AIDL direta é utilizável por app   | Não demonstrada                              |
| <code>/dev/vertex10</code> é utilizável sem root    | Negativa no teste realizado                  |
| NPU inteira pode executar um decoder transformer    | Não demonstrada                              |

## Proveniência

A atualização foi derivada do pacote `Exynos_NPU.zip` baixado em 2026-09-18. SHA-256 do ZIP: `873a0f8d3ff4d8e9a4168aafab0349f5becfa539cfc9b03d8a7033e3acb08af9`.

Os resultados públicos são derivados e sanitizados. Logs crus, comandos de coleta, binários compilados, bibliotecas vendor, firmware, modelos proprietários e dumps completos permanecem fora da distribuição pública.

# Capítulo incorporado: 31-npu-exynos-2400-evidence-map.md

## Mapa detalhado de evidências — NPU Exynos 2400

**Projeto:** Xclipse Open Project  
**Ramo:** plataforma NPU/NNAPI/ENN do Exynos 2400  
**Dispositivo de referência:** Samsung SM-S721B  
**Data da coleta:** 17–18/09/2026  
**Escopo:** documentar a plataforma observada, os caminhos de acesso, as provas de execução e os bloqueios. Este documento não descreve o XLIA nem implementa uma integração de aplicativo.

### 1. Leitura correta do resultado

A coleta não produziu uma única conclusão binária do tipo “a NPU está aberta” ou “a NPU está fechada”. Ela revelou **duas superfícies diferentes**.

A primeira é a superfície pública NNAPI. Um processo sem root descobriu `android.hardware.neuralnetworks.IDevice/enn`, enumerou o dispositivo ENN, compilou um grafo e executou `SOFTMAX` com readback numérico correto. Essa é uma rota confirmada.

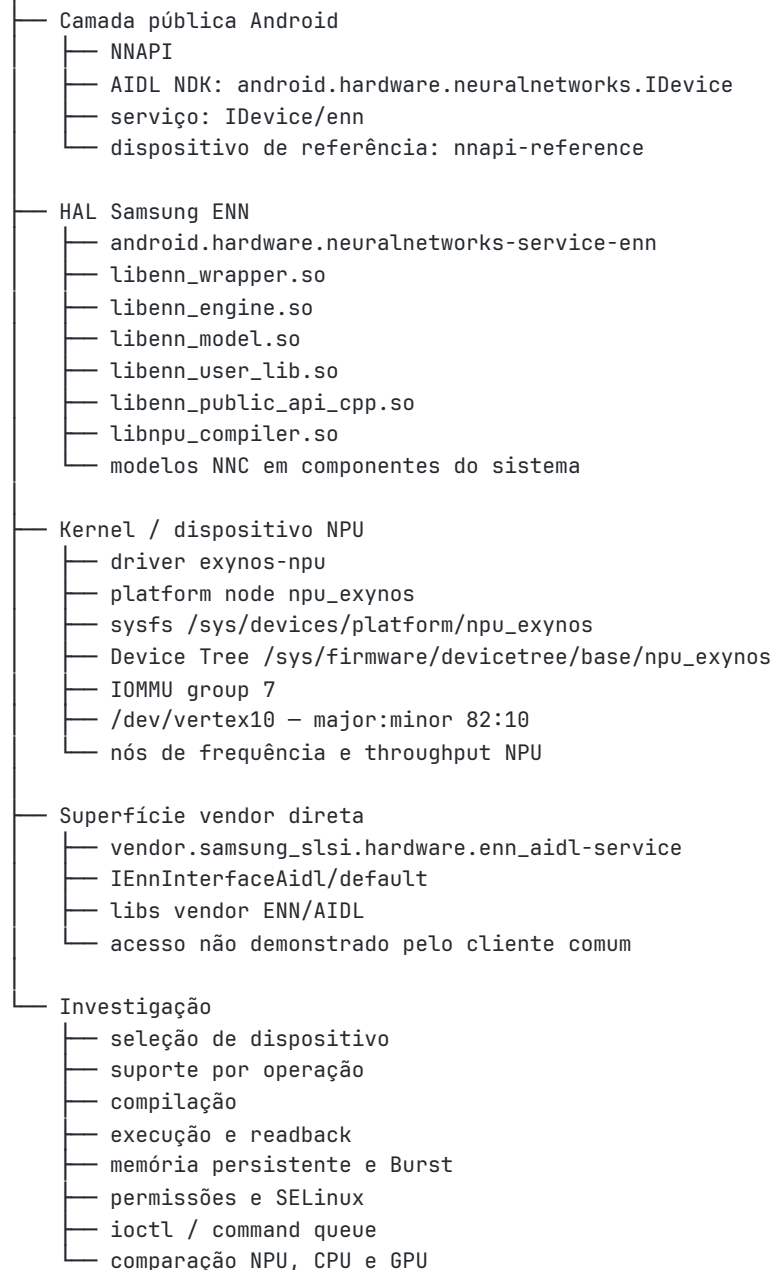
A segunda é a superfície vendor direta. O serviço `vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default` não foi encontrado pelos probes realizados. O

endpoint `/dev/vertex10` exige permissões que o UID comum não possui, e os `ioctl`s tentados com root não produziram uma operação funcional. Essa rota permanece bloqueada ou incompleta.

Portanto, o mapa deve mostrar simultaneamente uma **estrada pública funcional** e uma **estrada vendor direta com bloqueios**.

## 2. Mapa de arquitetura observada

Exynos 2400 / Android



### 3. Inventário dos experimentos

| Item | Tema                                  | Resultado                                                                                                            | Classe                               |
|------|---------------------------------------|----------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| 01   | Serviços Binder e bibliotecas         | <code>IDevice/enn</code> encontrado; bibliotecas ENN relacionadas enumeradas                                         | Probe confirmado                     |
| 02   | Interfaces Binder, AIDL, HIDL e VINTF | Manifesto <code>IDevice/enn</code> , HAL e arquivos de compatibilidade encontrados                                   | Evidência estrutural confirmada      |
| 03   | Permissões e SELinux                  | HAL no domínio <code>hal_neuralnetworks_service_enn_default</code> ; libs com rótulos vendor/HAL                     | Limite de segurança confirmado       |
| 03b  | Complemento SELinux                   | Arquivos de policy presentes; nenhuma regra pública suficiente para liberar o endpoint direto                        | Probe parcial                        |
| 04   | Dispositivos e endpoints              | <code>/dev/vertex10</code> identificado como dispositivo NPU                                                         | Probe confirmado                     |
| 04b  | Contexto do dispositivo               | <code>vendor_npu_device:s0</code> , <code>system:system</code> , major:minor <code>82:10</code>                      | Permissão observada                  |
| 08   | Rastreamento de aplicação real        | Processos de sistema e referências a <code>/sys/.../npu_exynos</code> ; serviço vendor direto não apareceu conectado | Observação parcial                   |
| 09   | UID normal versus root                | Ambos falharam ao localizar o AIDL vendor direto                                                                     | Resultado negativo específico        |
| 09b  | Polling do AIDL vendor                | Pollings repetidos retornaram nulo ou foram prejudicados por binário ausente/permissão de execução                   | Resultado misto; não generalizar     |
| 09c  | Diagnóstico de coleta                 | Uma rodada falhou porque o binário não estava presente                                                               | Falha de instrumentação              |
| 10   | Binder NNAPI HAL                      | Binder remoto <code>IDevice/enn</code> , ping <code>STATUS_OK</code> sem root                                        | Smoke test confirmado                |
| 11   | Suporte a SOFTMAX                     | <code>enn</code> reportou SOFTMAX suportado                                                                          | Suporte por operação confirmado      |
| 12   | Execução SOFTMAX                      | Compilação, execução e readback corretos sem root                                                                    | Execução confirmada                  |
| 13   | BATCH_MATMUL inicial                  | Modelo inválido                                                                                                      | Falha de montagem do teste           |
| 13b  | BATCH_MATMUL corrigido                | Operação não suportada; compilação ENN terminou com status 4                                                         | Limite do driver/compiler confirmado |
| 14   | Benchmark inicial                     | Modelo inválido                                                                                                      | Falha de montagem do teste           |
| 14b  | Benchmark corrigido                   | CPU executou; ENN não compilou                                                                                       | Comparação incompleta                |

| Item | Tema                       | Resultado                                                                                             | Classe                            |
|------|----------------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------|
| 15   | Abertura de vertex10       | UID comum recebeu <code>Permission denied</code>                                                      | Bloqueio rootless confirmado      |
| 16   | SELinux permissive e ioctl | Root abriu após <code>setenforce 0</code> ; ioctls retornaram <code>EINVAL / EFAULT</code>            | Probe privilegiado inconclusivo   |
| 17   | Descoberta de ioctl        | Não houve símbolo público claro nem comando funcional identificado                                    | Investigação aberta               |
| 17b  | Sysfs do vertex            | <code>/dev/vertex10</code> e <code>vertex11</code> aparecem; ligação com <code>/sys</code> confirmada | Topologia confirmada              |
| 18   | Sysfs e Device Tree        | Driver, IOMMU, governors, DVFS, command data e nós NPU expostos                                       | Arquitetura estrutural confirmada |

## 4. Serviços e interfaces

### 4.1 Serviço público confirmado

A listagem Binder encontrou:

```
android.hardware.neuralnetworks.IDevice/enn
```

O teste sem root obteve um Binder não nulo, confirmou `isRemote() = true` e recebeu `AIBinder_ping() = 0`, interpretado como `STATUS_OK`. O retorno `AIBinder_prepareTransaction() = -38` não invalida a descoberta; ele mostra apenas que aquela preparação específica não era suportada pelo caminho usado no probe.

### 4.2 VINTF e HAL

Os arquivos observados incluem:

```
/vendor/etc/vintf/manifest/android.hardware.neuralnetworks-service-enn.xml
/vendor/bin/hw/android.hardware.neuralnetworks-service-enn
/vendor/etc/init/android.hardware.neuralnetworks-service-enn.rc
/vendor/etc/init/enn-lazy.rc
/vendor/lib64/android.hardware.neuralnetworks@1.0.so
/vendor/lib64/android.hardware.neuralnetworks@1.1.so
/vendor/lib64/android.hardware.neuralnetworks@1.2.so
/vendor/lib64/android.hardware.neuralnetworks@1.3.so
```

O manifesto declara `IDevice/enn`. A propriedade observada indicou o serviço ativo:

```
init.svc.neuralnetworks_hal_service_enn = running
```

As matrizes de compatibilidade do sistema também mencionam `android.hardware.neuralnetworks` e `vendor.samsung_slsi.hardware.enn`.

### 4.3 Superfície vendor direta

O sistema contém o binário:

```
/vendor/bin/hw/vendor.samsung_slsi.hardware.enn_aidl-service
```

Também existe a biblioteca:

```
/vendor/lib64/vendor.samsung_slsi.hardware.enn_aidl-V1-ndk.so
```

Mesmo assim, o nome pesquisado abaixo retornou nulo nas rodadas de `checkService` :

```
vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default
```

A presença do binário e da biblioteca prova que a superfície existe no firmware. Não prova que ela esteja registrada, disponível para qualquer cliente ou autorizada para um aplicativo comum.

## 5. Bibliotecas e componentes observados

A coleta enumerou componentes de sistema e vendor associados ao ENN/NPU:

```
libenn_wrapper_system.so
libneural.snap.samsung.so
libneuralnetworks_packageinfo.so
libenn_common_utils.so
libenn_cpu_operators.so
libenn_engine.so
libenn_engine_lib.so
libenn_gc_vendor.so
libenn_klm_vendor.so
libenn_model.so
libenn_public_api_cpp.so
libenn_public_api_cpp_lib.so
libenn_seva_vendor.so
libenn_user.samsung_slsi.so
libenn_user_driver_cpu.so
libenn_user_driver_gpu.so
libenn_user_driver_gpu_lib.so
libenn_user_driver_unified.so
libenn_user_lib.so
libenn_wrapper.so
libnpu_compiler.so
libsait_npu_compiler.so
```

A coleta também observou `KnoxNeuralNetworkRuntime.apk` , o serviço HAL e arquivos `.nnc` associados a processamento de câmera. Esses elementos indicam que o sistema utiliza formatos e componentes específicos para modelos neurais. Eles não devem ser copiados para um aplicativo nem redistribuídos como parte do XOP.

## 6. SELinux, usuários e permissões

O processo HAL foi observado no contexto:

```
u:r:hal_neuralnetworks_service_enn_default:s0
```

O executável recebeu o rótulo:

```
u:object_r:hal_neuralnetworks_service_enn_default_exec:s0
```

O serviço vendor AIDL recebeu:

```
u:object_r:hal_enn_default_exec:s0
```

O endpoint direto recebeu:

```
u:object_r:vendor_npu_device:s0
```

O arquivo apareceu com dono e modo semelhantes a:

```
crw-r--r-- system system 82,10 /dev/vertex10
```

Apesar da aparência permissiva do modo Unix, o cliente comum recebeu `errno=13`, `Permission denied`. Isso demonstra que modo Unix, domínio SELinux e política efetiva precisam ser analisados juntos.

O UID comum observado foi `10415`, no domínio `u:r:untrusted_app_27:s0`. A comparação root utilizou UID 0 no domínio `u:r:ksu:s0`. A diferença de contexto é material: root não reproduz o ambiente de um aplicativo Android normal.

## 7. Endpoints, sysfs e topologia

A coleta estabeleceu a correspondência:

```
/dev/vertex10
  ↓ major:minor 82:10
/sys/class/vision4linux/vertex10
  ↓
/sys/devices/platform/npu_exynos/vision4linux/vertex10
  ↓
/sys/devices/platform/npu_exynos
  ↓
/sys/bus/platform/drivers/exynos-npu
```

O nó `vertex10` aponta para `npu_exynos`. A plataforma também possui `vertex11` em referências de descoberta, embora a investigação principal tenha usado `vertex10`.

O sysfs de `npu_exynos` expõe, entre outros, os seguintes grupos:

- `driver`, `subsystem`, `modalias` e `uevent`;
- `iommu` e `iommu_group`;
- `power` e `wakeup`;
- `qos_freq`;
- `afm_irp`, `afm_mode`, `afm_onoff`, `afm_restore_msec`, `afm_tdc_threshold` e `afm_tdt`;
- `log_level`, `npu_err_in_dmesg`, `suspend_resume_test` e `version`;
- `vision4linux`;
- nós de frequência e throughput relacionados ao NPU.

A existência de atributos de DVFS, QoS, AFM, wakeup e erro permite construir uma investigação térmica e de frequência. Ela não significa que um aplicativo comum possa escrever nesses atributos.

## 8. Device Tree observado

O nó foi identificado como:

```
name = npu_exynos
compatible = samsung,exynos-npu
OF_FULLNAME = /npu_exynos
DRIVER = exynos-npu
```

O Device Tree inclui referências a:

- `iommus`;
- `samsung,iommu-group`;
- `sysmmu,best-fit`;
- `dma-coherent` e `dma-window`;
- `interrupts`;
- `samsung,npumem-address` e `samsung,npumem-names`;
- `samsung,npurmem-address`;
- `samsung,npunode-names`, `samsung,npunode-num` e `samsung,npunode-freq`;
- `samsung,npusys-corenum`;
- `samsung,npusched-dvfs`, `samsung,npusched-names`, `samsung,npusched-min-active-cores` e `samsung,npusched-afmlimit`;
- `samsung,npugovernor-npufreq`, `samsung,npugovernor-intfreq` e `samsung,npugovernor-miffreq`;
- `samsung,npudvfs-open-clock`, `samsung,npudvfs-close-clock`, `samsung,npudvfs-open-dvfs`, `samsung,npudvfs-close-dvfs` e tabelas DVFS;
- `samsung,npuinter-isr-cpu-affinity`;
- dezenas de propriedades `samsung,npucmd-*` para controle de clock, DSP, DNC, GNPu, NPU memory e STM;
- `samsung,imgloader-s2mpu-support`;
- `vertex_name`.

Esses nomes revelam a riqueza da interface interna do driver. Eles não são uma API pública. A documentação os correlaciona com o comportamento observável por NNAPI sem tratar registradores ou atributos de produção como interface de aplicativo.

## 9. Execução NNAPI/ENN confirmada sem root

### 9.1 Descoberta

O teste encontrou dois dispositivos:

```
[0] nome=enn tipo=4  
[1] nome=nnapi-reference tipo=2
```

O primeiro é o dispositivo vendor ENN; o segundo é a referência CPU. O teste selecionou explicitamente `enn`.

### 9.2 Suporte por operação

Para `SOFTMAX`, o modelo terminou com status zero e `getSupportedOperationsForDevices` retornou sucesso. O dispositivo `enn` foi reportado como suportando a operação.

### 9.3 Compilação e execução

A sequência confirmada foi:

```
ModelFinish = 0  
CompilationCreateForDevices status = 0  
CompilationFinish status = 0  
Execution_compute status = 0
```

O resultado foi:

```
Output do driver: [0.032059, 0.087144, 0.236883, 0.643914]  
Softmax esperado: [0.032059, 0.087144, 0.236883, 0.643914]  
Diferença máxima: 0.000000
```

Como houve compilação, execução e comparação numérica, esta prova é mais forte que enumeração ou simples criação de um objeto NNAPI.

## 10. Matriz de operações e workloads

| Workload                             | ENN                                                                   | CPU/reference                                 | Interpretação                                        |
|--------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------|------------------------------------------------------|
| SOFTMAX pequeno                      | Compilou e executou; diferença máxima 0,000000                        | Resultado esperado igual                      | Execução ENN confirmada sem root                     |
| SOFTMAX de 1024 elementos            | Aproximadamente 1,0264 ms                                             | Aproximadamente 0,0104 ms                     | Funcionalidade não implica vantagem de desempenho    |
| Soma elementar                       | Resultado [11,22,33,44] em teste anterior                             | Resultado correto                             | Execução mínima observada                            |
| FULLY_CONNECTED INT8, quatro camadas | Aproximadamente 1,1148 ms; checksum igual                             | Aproximadamente 6,2407 ms                     | Melhor evidência de vantagem ENN nos grafos testados |
| FULLY_CONNECTED FP                   | Aproximadamente 1,2092 ms                                             | Aproximadamente 0,1332 ms                     | CPU mais rápida no workload observado                |
| BATCH_MATMUL inicial                 | Modelo inválido                                                       | Não comparável                                | Falha de montagem do modelo                          |
| BATCH_MATMUL corrigido               | getSupportedOperationsForDevices : não suportado; compilação status 4 | Não usado como comparação equivalente         | Bloqueio específico do ENN                           |
| Benchmark corrigido                  | create=0 , finish=4                                                   | 20 execuções em 207,13 ms; 10,356 ms/execução | Benchmark NPU incompleto porque ENN não compilou     |

Os números não devem ser transformados em uma afirmação geral de que a NPU é mais rápida que a CPU. Eles são específicos dos grafos, formas, tipos, tamanhos e estado do dispositivo.

## 11. Investigação do endpoint vertex10

O cliente sem root tentou:

```
open(O_RDONLY)
open(O_RDWR)
```

Ambas as tentativas retornaram `errno=13` , `Permission denied` .

Com root e `setenforce 0` temporário, o probe conseguiu abrir o endpoint e mapear memória:

```
open(O_RDONLY) fd=3
mmap() = sucesso
```

Entretanto, os comandos testados retornaram:

```
ioctl(cmd=0) = EINVAL
ioctl(cmd=1) = EINVAL
ioctl(cmd=2) = EFAULT
ioctl(cmd=3) = EINVAL
ioctl(cmd=4) = EINVAL
ioctl(cmd=5) = EINVAL
```

O retorno mostra que abrir e mapear não equivale a conhecer o protocolo do driver. A investigação de strings encontrou referências como `REG_BASE_ADDR_CMDQ_CORE`, `runCMDQGenerator`, `runISAGenerator`, `NCP_BINARY`, `NCP Version` e `ioctl`, mas não forneceu uma tabela pública de comandos utilizável.

A ausência de headers em `/system` e `/vendor` para `vertex` ou `NPU_IOCTL` também impede afirmar uma ABI estável.

## 12. Serviço vendor: por que a rota falhou

O teste `checkService` procurou repetidamente:

```
vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default
```

As rodadas retornaram nulo. Houve também polling de 30 segundos enquanto a câmera estava aberta e ativa, sem obter um serviço funcional. Algumas rodadas adicionais foram inválidas por falhas de instrumentação: binário ausente, execução em local sem permissão ou tentativa de executar arquivo em armazenamento externo.

Essas falhas precisam ser separadas em duas categorias:

1. **Falha real observada:** o serviço não foi encontrado pelo cliente nas rodadas válidas.
2. **Falha de coleta:** algumas rodadas não chegaram a executar o probe correto.

A conclusão segura é negativa apenas para a rota válida testada naquele estado do aparelho. Não é correto afirmar que o serviço vendor não existe, pois o binário e a biblioteca estão presentes no firmware.

## 13. Caminhos verdes, bloqueios e áreas abertas

### Caminhos confirmados

- NNAPI pública com dispositivo `enn`.
- Processo sem root obtendo Binder remoto.
- Consulta de suporte de operação.
- Compilação de `SOFTMAX` pequeno.
- Execução com readback correto.
- Device Tree e sysfs do driver `exynos-npu`.
- Identificação de `/dev/vertex10` e major:minor `82:10`.

### Bloqueios confirmados

- Acesso comum direto a `/dev/vertex10`.
- Suporte a `BATCH_MATMUL` no formato testado.
- Serviço AIDL vendor direto não descoberto.

- ABI de `ioctl` não identificada.
- Benchmark ENN versus CPU incompleto para o modelo corrigido.
- Algumas rodadas de polling invalidadas por problemas de execução do binário.

## Áreas de investigação

- `ANeuralNetworksBurst` e redução do overhead de submissão.
- Memória compartilhada e lifetime de operandos.
- Reuso de compilação e execução persistente.
- Particionamento por operação com fallback explícito.
- Formas alternativas de matriz e tipos suportados.
- Correlacionamento de sysfs, frequência, erro e tempo por execução.
- Registro condicional do serviço vendor em estados específicos do sistema.
- Relação entre ENN, `libnpu_compiler`, NCP e command queue, sem acesso direto destrutivo.

## 14. O que não está provado

A coleta não prova que:

- todo operador NNAPI seja executado na NPU;
- toda execução `enn` seja mais rápida que CPU;
- o dispositivo `enn` não use fallback interno para uma operação específica;
- um decoder transformer completo seja aceito;
- o endpoint `vertex10` seja uma ABI pública;
- o serviço vendor AIDL possa ser usado por qualquer aplicativo;
- a presença de modelos `.nnc` permita reutilização por terceiros;
- a escrita em sysfs ou o uso de root transforme a interface interna em API estável.

## 15. Arquitetura consolidada observada

### 15.1. Camadas de software

A arquitetura observada começa na aplicação ou cliente NNAPI. O cliente não conversa diretamente com registradores, firmware ou `vertex10`. Ele constrói um modelo NNAPI, consulta suporte, solicita compilação para `IDevice/enn` e entrega buffers de operandos ao runtime. O HAL `android.hardware.neuralnetworks-service-enn` atua como fronteira entre a interface Android e o software Samsung ENN.

Dentro da pilha ENN aparecem camadas distintas. `libenn_wrapper` e `libenn_wrapper_system` fazem a ponte de integração. `libenn_model` mantém a representação do grafo. `libenn_engine` e `libenn_engine_lib` participam da preparação e execução. `libenn_user_lib` e `libenn_user.samsung_slsi` conectam o runtime ao driver vendor. Os drivers separados de CPU, GPU e unified indicam que o runtime possui mais de um caminho de execução, portanto o nome `enn` sozinho não revela qual subcomponente executou cada operação.

### 15.2. Modelo, compilador e formato interno

As strings das bibliotecas mostram `NCP_BINARY`, `NCP_Version`, `NCPBuffer`, `NPUCompiler`, `NPUCommon`, `backends`, `NPUcrane`, `NPUdove`, `NPUeagle` e `NPUroot`, além de geradores `runNcpGenerator`, `runCMDQGenerator` e `runISAGenerator`. Isso indica uma cadeia de transformação que parte de um grafo e chega a uma representação NCP/CMDQ específica do acelerador.

Essa evidência é estática. Ela demonstra a presença de conceitos de compilação, command queue e geração de ISA no software instalado, mas não revela a codificação completa dos comandos nem autoriza a montagem de uma fila privada.

### 15.3. Hardware lógico e blocos internos

O Device Tree nomeia blocos `gnpu0`, `gnpu1`, `snpu0`, `snpu1`, `dnc`, `dsp` e `npumem`. As propriedades `samsung,npucmd-*` incluem ativação e desativação de clocks, DSP, NPU, DNC, STM e caminhos de acesso a registradores. Os nomes `sfrgnpu0`, `sfrgnpu1`, `sfrsnpu0`, `sfrsnpu1`, `sfrdnc`, `sfrdsp0` e `sfrnpumem` aparecem nas tabelas de controle observadas.

O mapa também registra `NPU0` e `NPU1` em nomes de nó, frequência, scheduler e DVFS. Isso indica uma arquitetura com mais de um domínio lógico de processamento, mas não permite concluir que ambos possam ser selecionados individualmente pela API pública.

### 15.4. Memória, IOMMU e DMA

O nó `npu_exynos` possui referências a `iommu`, `samsung,iommu-group`, `sysmmu,best-fit`, `dma-window`, `dma-coherent`, `samsung,npumem-address`, `samsung,npumem-names` e `samsung,npumem-address`. A topologia observada liga o dispositivo a um grupo IOMMU e a dois fornecedores SysMMU em `sysfs`.

Esses elementos mostram que operandos e buffers passam por uma política de endereçamento e tradução própria. O mapa não contém uma prova de que um buffer alocado arbitrariamente por um aplicativo possa ser convertido em memória NPU pelo endpoint direto. A rota pública NNAPI esconde essa operação dentro do HAL e do runtime.

### 15.5. Frequência, QoS, AFM e energia

O `sysfs` expõe `qos_freq`, governos de NPU, frequências de `npu_freq`, `int_freq` e `mif_freq`, tabelas DVFS, limites de atividade, número mínimo de cores, afinidade de interrupção e propriedades AFM como `afm_irp`, `afm_mode`, `afm_onoff`, `afm_restore_msec`, `afm_tdc_threshold` e `afm_tdt`.

Também aparecem nós de throughput para `NPU0`, `NPU1` e NPU agregado. A presença dessa infraestrutura explica por que tempo de execução, frequência e temperatura precisam ser interpretados como parte do estado do sistema. Ela não é uma autorização para escrever em `sysfs` ou forçar frequências.

### 15.6. Firmware, wakeup e recuperação

O sistema mantém propriedades de `wakeup` para o dispositivo `npu_exynos`, incluindo contadores de atividade, mudanças, eventos, expirações e tempo ativo. O `sysfs` também expõe `npu_err_in_dmesg`, `log_level`, `suspend_resume_test` e `version`.

Esses nós representam a integração do acelerador com suspensão, recuperação e diagnóstico do kernel. O mapa confirma a existência dos mecanismos de controle e observabilidade; não confirma o protocolo de recuperação nem a semântica de cada comando.

### 15.7. Fronteira entre API pública e ABI interna

A API pública é identificável por `IDevice/enn`, operações NNAPI, compilação e execução. A ABI interna aparece nos binários vndor, no endpoint `vertex10`, nos símbolos NCP/CMDQ e nas propriedades Device Tree. Entre as duas existe uma fronteira deliberada de HAL, namespaces, SELinux, permissões de dispositivo e formatos proprietários.

O fato de o caminho NNAPI funcionar sem root e o caminho `vertex10` falhar para UID comum é uma evidência dessa separação. O XOP deve documentar essa fronteira, não tratá-la como uma simples ausência de comando.

## 16. Proveniência e classificação

Fonte: pacote `Exynos_NPU.zip`, recebido em 18/09/2026. SHA-256 do pacote:  
`873a0f8d3ff4d8e9a4168aafab0349f5becfa539cfc9b03d8a7033e3acb08af9`.

Os itens 10 e 12 constituem a evidência de execução mais forte. Os itens 13b, 15, 16 e 17 constituem evidência de limites ou investigação incompleta. Os itens 09b e 09c precisam ser mantidos como falhas de instrumentação, não como prova de comportamento do hardware.

Logs crus, comandos de coleta, binários compilados, bibliotecas vendor, firmware, modelos proprietários, dumps e informações pessoais não fazem parte da distribuição pública.

---

# Capítulo incorporado: 5-dois-meios-essenciais.md

## 5 dois meios — 10 objetivos atuais da Xclipse 940

**Projeto interno:** XO940 **Hardware:** Samsung Xclipse 940 no SM-S721B **Regra:** os cinco primeiros objetivos são bloqueadores técnicos. Os cinco seguintes são descobertas rápidas, de baixo risco e úteis para acelerar o trabalho.

### O que estamos tentando construir

O objetivo de longo prazo é descobrir se a Xclipse 940 pode receber um **driver de espaço de usuário independente**, com uma arquitetura comparável à de um driver Mesa/Vulkan ou a uma camada customizada que converse com o kernel SGPU existente. Isso não significa assumir que a Xclipse é uma AMDGPU de PC, nem que o caminho implementação Vulkan de referência possa ser copiado. A árvore Device Tree, a UAPI SGPU, os firmware, a VM, os rings e a ABI Android precisam ser tratados como contratos próprios.

A evidência atual mostra um caminho vendor real de submissão GFX, mas ainda não mostra que um cliente novo consiga controlar esse caminho e validar sua saída. Portanto, o projeto deve avançar da plataforma para o kernel, do kernel para memória/submissão, e só depois para Vulkan, shader e compiler.

### Os cinco objetivos mais importantes

#### C1 — Fechar o contrato Device Tree → kernel SGPU → plataforma

Documentar completamente o nó `sgpu@22200000`, regiões `gpu`, `doorbell`, `debug`, `pwrctl`, `sysreg` e `htu`, IRQs `SGPU / GPU-AFM`, `G3DCORE`, `clocks`, `DVFS`, `reset`, `AFM/IFPO`, `DMA-coherent` e variantes `EVT0`.

**Saída:** mapa reproduzível ligando cada propriedade da Device Tree ao subsistema kernel que a consome. Sem essa saída, falhas de energia, clock ou reset podem ser confundidas com falhas do driver.

#### C2 — Reproduzir o ciclo de vida de memória do cliente

Fechar o caminho `BO/GEM/TTM/DMA-BUF` → `heap` → `IOMMU/SysMMU` → `VA` → `permissões` → `cache` → `flush` → `unmap`. O objetivo é diferenciar “a CPU conseguiu mapear” de “a GPU conseguiu acessar corretamente”.

**Saída:** tabela controlada de alocação, mapeamento, sincronização, erro e limpeza, incluindo `page size`, `alinhamento`, `residency` e `buffers` protegidos quando aplicável.

#### C3 — Isolar e correlacionar um job GFX controlado

Usar a evidência já observada de `amdgpu_cs_ioctl`, `amdgpu_sched_run_job`, `amdgpu_ib_schedule`, `gfx_0.0.0`, `context` e `seqno` para separar um workload conhecido das atividades do compositor, câmera, Chrome e RenderThread.

**Saída:** um job próprio com IB conhecido, contexto identificado, `fence/seqno` correlacionados e `buffer` de resultado validado. Agendamento de job sem resultado não basta.

#### C4 — Definir a fronteira entre kernel aberto, ABI vendor e cliente independente

Mapear quais `ioctls`, `structs`, `flags`, `handles`, `permissões`, `namespaces` e `bibliotecas` são realmente necessários para um cliente novo. Separar a UAPI SGPU pública ou observável das interfaces internas e das dependências proprietárias.

**Saída:** uma tabela de contrato mínimo com origem, licença, estabilidade, processo consumidor e risco de reutilização. Esse é o ponto que decide se a rota deve aproveitar o kernel existente ou exigir uma adaptação adicional.

## C5 — Demonstrar o primeiro workload independente, começando por compute

Tentar um caminho mínimo e reversível: identificar dispositivo, abrir o caminho permitido, criar contexto, alocar entrada/saída, submeter uma operação simples e validar readback. OpenCL vendor pode servir como referência de comportamento, mas não deve ser tratado como ABI pública automaticamente.

**Saída:** entrada conhecida → execução Xclipse 940 → saída conhecida , ou um relatório preciso dizendo em qual camada o caminho foi bloqueado. P5.04, por si só, foi uma captura negativa e não cumpre este objetivo.

## As cinco descobertas mais fáceis e rápidas

### Q1 — Comparar automaticamente todas as variantes Device Tree SGPU

Gerar uma tabela entre `s5e9945-sgpu_common.dtsi` , `s5e9945-sgpu_evt0.dtsi` e includes relacionados, destacando diferenças de `reg` , IRQ, clocks, power-domain, IOMMU, DMA e revisão.

**Por que é rápida:** é trabalho de fonte e inventário, sem submeter GPU. **Resultado:** reduz o risco de usar a variante errada no primeiro driver.

### Q2 — Construir um inventário único do runtime SGPU

Consolidar nós DRM, sysfs, firmware, frequência, módulos, permissões, processos e FDs para `renderD128` , mantendo timestamps e identidade do dispositivo.

**Por que é rápida:** os dados já aparecem em R1/R2/P2. **Resultado:** elimina coletas duplicadas e mostra quais condições mudam entre sessões.

### Q3 — Mapear bibliotecas e namespaces da cadeia vendor

Catalogar caminho, arquitetura ELF, SONAME, NEEDED, BuildID, namespace e processo para `vulkan.samsung.so` , `libdrm_sgpu.so` , `libOpenCL.so` , `libSGPUOpenCL.so` e `libvulkan.so` .

**Por que é rápida:** é análise estática e de runtime, sem publicar os binários. **Resultado:** identifica a primeira barreira real para um cliente externo.

### Q4 — Indexar a UAPI e os símbolos por subsistema

Criar índices separados para GEM/BO, VM/IOMMU, CS/IB/ring, fences/syncobjs, firmware/reset, GFX/COMPUTE/SDMA e DVFS/debug, marcando cada símbolo como interface, implementação, configuração, log ou hipótese.

**Por que é rápida:** aproveita a fonte Samsung já inventariada. **Resultado:** acelera a localização das funções correspondentes aos eventos de runtime.

### Q5 — Isolar a assinatura de um job real no trace

Escolher uma janela P4.5/P4.6/P4.12 e seguir `sched_job` , `context` , `seqno` , PID, `num_ibs` , ring, VM flush e eventos de limpeza. Não é necessário ainda interpretar o IB.

**Por que é rápida:** os traces já contêm grande parte dos identificadores. **Resultado:** produz o primeiro modelo de causalidade sem fingir que houve readback.

## Ordem de execução

```
Q1 + Q2 + Q3 + Q4 + Q5
  ↓
C1: Device Tree e plataforma
  ↓
C2: memória, IOMMU e VM
  ↓
C3: job GFX isolado e conclusão
  ↓
C4: contrato UAPI/ABI e ponte Android
  ↓
C5: primeiro workload independente
  ↓
shader / ISA / compiler / Vulkan
```

## Critério contra falsos positivos

Um nome contendo `test`, `probe`, `trace`, `compute` ou `submit` não classifica o arquivo como teste bem-sucedido. Inicializador, consulta de capacidade, descoberta de tracepoint, criação de pipeline sem dispatch e trace sem resultado são evidências parciais. Só promovemos uma operação quando sua execução, contexto e saída são observáveis.

## Decisão técnica atual

A Xclipse 940 não deve ser tratada como uma simples GPU de PC transplantada para um celular. O caminho observado combina elementos familiares do ecossistema AMD/AMDGPU — BO, VM, scheduler, IB, rings e nomenclatura de traces — com integração Samsung específica de Device Tree, power-domain, IOMMU, firmware, Android, namespaces e bibliotecas vendor. Isso torna uma arquitetura de driver independente plausível como investigação, mas impede copiar diretamente um driver existente sem validar cada contrato.

Os artefatos crus, testes, scripts de captura, bibliotecas e binários permanecem fora do GitHub público. Este documento publica apenas objetivos, critérios e interpretação técnica.

## Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

# Capítulo incorporado: estudo-detalhado-xclipse-940-2026-09-06.md

## Estudo detalhado da Xclipse 940 — evidências de runtime, Device Tree e rotas de desenvolvimento

**Projeto interno:** XO940 **Hardware analisado:** Samsung Xclipse 940 no dispositivo SM-S721B **Data da revisão:** 2026-09-06  
**Autor:** Manus AI

### Conclusão executiva

O pacote novo acrescenta evidência substancial sobre o caminho real da Xclipse 940. A coleta de runtime expõe um nó `sgpu@22200000` no Device Tree, um domínio de energia G3DCORE, regiões de memória reservada, DMA heap de GPU, caminhos de IOMMU/SysMMU e propriedades de DVFS, IRQ e registradores. Esse conjunto é útil para portar a descrição de plataforma e para entender o contrato necessário antes de qualquer driver de espaço de usuário.

Os traces P4.5, P4.6 e P4.12 são a evidência mais importante desta rodada. Eles mostram chamadas `amdgpu_cs_ioctl`, execução pelo scheduler em `gfx_0.0.0` e `amdgpu_ib_schedule`, incluindo `num_ibs=3`. P4.5 também mostra o vínculo temporal entre `sgpu_pio_map_queue`, `amdgpu_vm_bo_cs`, `amdgpu_vm_flush` e o envio do job. Isso confirma atividade de submissão GFX no caminho observado.

A evidência histórica não demonstrava um workload controlado; os logs de 2026-09-07 agora mostram que um cliente próprio obteve aceitação de um CS específico, mas ainda não demonstram execução GPU, fence própria ou readback. P5.04 não capturou um IB compute nem eventos de dispatch. A presença de exports OpenCL, de `LibSGPUOpenCL.so` e de símbolos de compilador prova uma superfície instalada e um backend proprietário existente, mas não prova que qualquer programa externo possa usar essa superfície fora dos namespaces e permissões originais.

A rota tecnicamente mais promissora é incremental. Primeiro, deve-se reproduzir o contrato de plataforma e memória com base no Device Tree e na UAPI SGPU. Depois, deve-se correlacionar submissão, VM, fences e conclusão em uma aplicação controlada. Somente após isso faz sentido estudar a tradução de comandos, ISA e uma camada Vulkan.

### 1. Regra de evidência

Os nomes dos arquivos e os títulos das etapas não foram tratados como resultados. Cada afirmação foi classificada por sua observação concreta:

| Classe          | Significado                                                          | Uso nesta revisão                                                  |
|-----------------|----------------------------------------------------------------------|--------------------------------------------------------------------|
| Observado       | aparece em saída de runtime, trace, inventário ELF ou Device Tree    | pode sustentar uma conclusão factual limitada                      |
| Corroborado     | aparece em mais de uma coleta independente                           | pode orientar uma prioridade de implementação                      |
| Indício         | símbolo, caminho, string ou estrutura sem execução controlada        | orienta investigação, não prova capacidade                         |
| Não demonstrado | o arquivo foi executado, mas a saída não contém a operação procurada | deve bloquear conclusões otimistas                                 |
| Roteiro         | instrução, inicializador, comando ou preparação                      | não é resultado e não deve ser apresentado como teste bem-sucedido |

P4.04, P5.04 e arquivos de descoberta de tracepoints foram tratados com cuidado. A existência de seções “TRACE START” e “TRACE RESULT” não significa que o workload-alvo tenha sido executado. Em particular, P5.04 possui apenas atividade de

`tracing_mark_write` e não contém `amdgpu_cs_ioctl`, `amdgpu_sched_run_job`, `amdgpu_ib_schedule`, `dispatch` ou `readback`.

## 2. Device Tree e plataforma

O mapa de runtime identifica o nó principal em `/sys/firmware/devicetree/base/sgpu@22200000`. O compatible observado é `samsung-sgpu,samsung-sgpu`. O nó possui os filhos `gpu_pm`, `gpu_doorbell`, `gpu_debug`, `gpu_smntarg` e `gpu_sysreg`. Os nomes de região observados são `gpu`, `doorbell`, `debug`, `pwrctl`, `sysreg` e `htu`.

As interrupções são nomeadas `SGPU` e `GPU-AFM`. O nó referencia o phandle `0xc3`, que corresponde ao domínio `pd_g3dcore@0` / `pd_g3dcore`. O domínio foi observado como `samsung,exynos-pd` com estado `okay`. Essa relação é importante porque inicializar a GPU sem o domínio `G3DCORE`, sem clocks e sem reset coerentes pode produzir falhas que parecem de MMU ou de comandos, mas são de plataforma.

A arquitetura publicada separa o caminho da GPU dos demais consumidores do SoC. Ela inclui o nó `SGPU`, `G3D`, `BTS G3D`, memória reservada, DMA heap, grupos IOMMU e vínculos térmicos. Todos esses detalhes estão consolidados neste documento único.

## 3. Memória, VM e IOMMU

A coleta P2.10 registra 119.929 eventos `amdgpu_vm_pte_pde`, 11.307 `amdgpu_vm_set_ptes`, 10.998 `amdgpu_vm_update_ptes`, 1.947 `amdgpu_vm_flush`, 262 criações de BO e 262 movimentos de BO. Isso é evidência forte de que o caminho observado realiza gerenciamento de objetos e atualização de tabelas de página durante atividade gráfica.

P4.5 e P4.6 ligam a memória à submissão. Antes de alguns jobs aparecem `sgpu_pio_map_queue_entry` e `sgpu_pio_map_queue_exit`; durante o job aparecem `amdgpu_vm_bo_cs` e `amdgpu_vm_flush`; em seguida aparecem `amdgpu_sched_run_job` e `amdgpu_ib_schedule`. Essa sequência é um mapa operacional melhor do que uma lista isolada de símbolos.

A existência de endereços de page directory, PTEs e flags nos traces não revela, por si só, o formato completo das tabelas, a política de cache, a semântica de todos os bits ou o mecanismo de recuperação de fault. Esses itens continuam como trabalho de engenharia reversa controlada.

## 4. Submissão GFX, scheduler e fences

P4.12 contém uma janela com 13 chamadas `amdgpu_cs_ioctl`, 13 `amdgpu_sched_run_job` e 13 `amdgpu_ib_schedule`. Os eventos usam `timeline=gfx_0.0.0`, `ring_name=gfx_0.0.0` e `num_ibs=3`. P4.6 confirma a mesma cadeia junto com `amdgpu_vm_bo_cs`, `amdgpu_vm_flush`, `amdgpu_ib_pipe_sync` e `sgpu_job_dependency` habilitados.

O que está comprovado é a passagem de jobs gráficos pelo caminho de submissão do driver observado. O que ainda não está comprovado é a leitura direta do conteúdo dos IBs, a interpretação dos pacotes pelo hardware, o sinal de fence no mesmo evento e o resultado de um buffer controlado. Os eventos clássicos de fence `amdgpu_fence*` estavam indisponíveis em P4.12; portanto, não se deve escrever “fence concluída” somente porque um job foi agendado.

A próxima coleta deve correlacionar o mesmo `sched_job`, `context` e `seqno` entre `ioctl`, `scheduler`, `IB`, `DMA fence` e evento de conclusão. Se o kernel não expuser todos os eventos, deve-se usar uma combinação documentada de tracepoints disponíveis, estado de ring e um buffer de resultado com assinatura conhecida.

## 5. Caminho Android, Vulkan e OpenCL

O inventário ELF confirma a presença de `vulkan.samsung.so`, `libdrm_sgpu.so`, `libOpenCL.so` e `libSGPUOpenCL.so` no ambiente fornecido. A cadeia de runtime também mostra processos de câmera, `surfaceflinger` e bibliotecas vendor carregadas em namespaces Android específicos. Isso confirma que o firmware da Samsung possui consumidores reais da GPU e uma pilha proprietária funcional em seus próprios limites.

A presença de exports OpenCL como `clEnqueueNDRangeKernel`, `clEnqueueReadBuffer`, `clBuildProgram` e `clCreateProgramWithIL` é uma oportunidade de estudo. Ela não prova que o backend possa ser reutilizado por um novo loader, porque símbolos exportados podem depender de inicialização privada, propriedades de contexto, permissões SELinux, bibliotecas auxiliares e formatos internos.

P5.01 identifica bibliotecas OpenCL e seus exports, mas não fecha um dispatch compute independente. P5.04 não contém o dispatch procurado. A conclusão correta é: **há uma superfície OpenCL proprietária instalada e há uma rota de investigação; a execução compute controlada ainda não foi demonstrada nesta coleta.**

## 6. Cinco prioridades críticas

| ID | Prioridade                                            | Por que é crítica                                                                        | Critério de saída                                                                              |
|----|-------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| C1 | Reproduzir o contrato Device Tree e power/clock/reset | sem plataforma estável, os demais resultados ficam ambíguos                              | mapa de recursos, IRQ, power-domain, clocks e reset reproduzido em documentação e probe mínimo |
| C2 | Fechar memória, IOMMU e VM                            | todo driver precisa alocar, mapear, sincronizar e desfazer BOs corretamente              | sequência controlada de BO → map → VM update → flush → unmap com erro e limpeza observáveis    |
| C3 | Fechar a submissão GFX e a conclusão                  | já há evidência de scheduling, mas falta um workload controlado com conclusão inequívoca | job próprio com IB conhecido, fence correlacionada e buffer de saída validado                  |
| C4 | Identificar o contrato UAPI/ABI mínimo                | símbolos e paths não substituem ioctl e estruturas compatíveis                           | tabela de ioctls, structs, flags, alinhamentos e permissões necessários para um cliente mínimo |
| C5 | Obter um caminho compute controlado                   | OpenCL vendor indica oportunidade, mas P5.04 não executou compute                        | dispatch simples com entrada/saída conhecida, ou prova objetiva do bloqueio e de sua camada    |

## 7. Cinco descobertas rápidas

| ID | Descoberta rápida                                                        | Evidência já disponível                                                                                                                | Utilidade                                                 |
|----|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Q1 | Enumerar exatamente os <code>reg-names</code> e os intervalos do nó SGPU | R1 lista <code>gpu</code> , <code>doorbell</code> , <code>debug</code> , <code>pwrcctl</code> , <code>sysreg</code> e <code>htu</code> | reduz erros no primeiro mapa de MMIO                      |
| Q2 | Correlacionar os três <code>num_ibs=3</code> com contexto e sequência    | P4.12 já fornece <code>sched_job</code> , <code>context</code> , <code>seqno</code> e <code>ring</code>                                | cria um identificador de job para as próximas coletas     |
| Q3 | Catalogar os tracepoints SGPU disponíveis                                | P5.05 lista eventos de PIO, devfreq, IFPO e dependências                                                                               | permite capturas menores e mais informativas              |
| Q4 | Mapear namespaces e dependências do OpenCL vendor                        | P3.9.x e R5 listam bibliotecas, SONAME e NEEDED                                                                                        | revela onde uma tentativa externa seria bloqueada         |
| Q5 | Comparar snapshots de VM por PID e por workload                          | P2.09, P2.10 e P2.11 já separam VM, PTE e PID                                                                                          | diferencia atividade de RenderThread, câmera e compositor |

## 8. Falhas e rotas de investigação

A principal falha observável não é uma ausência de GPU. É a ausência de uma fronteira pública e independente entre a pilha proprietária e um cliente novo. O Android possui nós DRM, bibliotecas vendor, namespaces e políticas de segurança, mas os relatórios R2 não identificam um caminho de cliente genérico com todos os atributos disponíveis.

A segunda falha é metodológica: muitos traces mostram atividade real do sistema, porém o workload não é isolado. `RenderThread` , `GrallocUploadTh` , Chrome, câmera e compositor podem gerar jobs simultâneos. Sem PID, context e seqno controlados, um evento real pode ser atribuído à operação errada.

A terceira falha é a falta de um resultado compute controlado. A rota OpenCL deve ser estudada como camada de observação e como referência de comportamento, não como API automaticamente reutilizável. A investigação deve começar por um programa mínimo que identifique plataforma, dispositivo, contexto, fila, kernel e leitura de um buffer; cada fase precisa de um marcador e de uma saída verificável.

## 9. O que não foi publicado

O repositório não publica os testes, scripts de captura, traces crus, dumps de memória, bibliotecas `.so` , binários ELF, ZIPs internos, código proprietário ou arquivos que permitam baixar diretamente o material bruto do dispositivo. O GitHub recebe apenas documentação derivada, hashes quando necessários, inventários sanitizados, mapas técnicos, relatórios de estudo e PDFs.

## 10. Referências internas de evidência

Os identificadores abaixo apontam para os artefatos recebidos na análise local. Os artefatos crus permanecem fora do repositório público, mas os nomes são preservados para auditoria privada.

| ID | Artefato analisado                                                                                                                                                                       | Tipo de evidência                              |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| E1 | <code>R1_Device_Tree_SGPU.txt</code> , <code>R1_power_iommu.txt</code> , <code>R1_g3d_links.txt</code>                                                                                   | Device Tree e plataforma                       |
| E2 | <code>R2_Runtime_Inventory_MASTER.txt</code>                                                                                                                                             | inventário runtime e nós DRM                   |
| E3 | <code>R3_MASTER.txt</code> , <code>R5_ICD_ELF_INVENTORY.txt</code>                                                                                                                       | bibliotecas, ELF e dependências                |
| E4 | <code>P2_10_RESUMO.txt</code> , <code>P2_11_PID_VM_CORRELATION.txt</code>                                                                                                                | VM, PTE, BO e PID                              |
| E5 | <code>P4_5_SGPU_LIVE_JOB_SUBMISSION.txt</code> ,<br><code>P4_6_SGPU_JOB_COMPLETION_FENCE_TRACE.txt</code> ,<br><code>P4_12_SGPU_FENCE_SIGNAL_JOB_RETIRE_RING_RPTR_CORRELATION.txt</code> | submissão, scheduler e IB                      |
| E6 | <code>P5_01_COMPUTE_PATH.txt</code> , <code>P5_04_COMPUTE_IB_CAPTURE.txt</code> ,<br><code>P5_05_TRACEPOINT_DISCOVERY.txt</code>                                                         | compute, negativa de captura e observabilidade |
| E7 | <code>P3_9.34_FINAL_RUNTIME_CHAIN.txt</code>                                                                                                                                             | cadeia Android e namespaces                    |

## Referências

13

Este estudo descreve evidências fornecidas pelo usuário e não afirma compatibilidade, segurança ou funcionalidade de um driver ainda não implementado.

## Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

## Addendum 2026-09-08 — coleta de reprodução

O pacote coleta de reprodução acrescentou um probe DRM direto em C. No SM-S721B, `card0` e `renderD128` reportaram `amdgpu`, enquanto `card1` e `renderD129` reportaram `exynos-drmdpu`. Esse resultado é útil para a topologia DRM, mas não prova GPU AMD física, compatibilidade AMDGPU upstream ou compatibilidade implementação Vulkan de referência. O pacote também demonstrou que o armazenamento compartilhado Android pode ser `noexec`, estabelecendo uma regra de reprodução: executar em workspace privado e arquivar em `armazenamento compartilhado`.

# Capítulo incorporado: initial-evidence.md

## Initial Evidence Review

**Evidence set:** Quick Share package `Tudo sobre a xclipse.zip` **Archive SHA-256:**

`cd73e2fa24ac083b9babf243b39772065b060abc9cd30fb476acabe74433e67f` **Review date:** 2026-09-05 **Target hardware:** Samsung SM-S721B / Xclipse 940 **Project:** Xclipse Open Project (internal name: XO940)

## Executive conclusion

The supplied package contains a coherent bring-up record, a Samsung kernel/platform source release, raw device logs, a DRM probe, a Vulkan pipeline-properties probe, vendor binaries, and continuity reports. The strongest current result is **DRM/KMD and memory bring-up**, not a working open driver or a compute execution test.

The SGPU probe opens `/dev/dri/renderD128`, identifies the platform driver as `sgpu`, queries device information, hardware IP counts, firmware versions, and page faults, creates a 64 KiB GTT GEM object, maps and touches it from the CPU, maps and unmaps a GPU virtual address, and closes the object. Its source says `Submits NOTHING to the GPU`. The observed `DONE fails=2` is therefore a probe result with two failed queries, not a failed compute test.

The Vulkan probe opens the system loader, creates a Vulkan instance, enumerates physical devices, looks for vendor `0x144d`, creates a device and a trivial compute pipeline if the Samsung device is visible, and queries `VK_KHR_pipeline_executable_properties`. It does not call `vkGetDeviceQueue`, `vkQueueSubmit`, `vkCmdDispatch`, `vkCmdDraw`, or perform a readback. The line `compute pipeline ok` would mean pipeline creation succeeded, not that a compute dispatch ran.

In the supplied execution, Vulkan exposed only `llvmpipe` with vendor `0x10005`; Samsung vendor `0x144d` was not visible. Directly loading `/vendor/lib64/hw/vulkan.samsung.so` from the Termux namespace was blocked and ended in a segmentation fault. This makes the Android loader/namespace/ICD boundary the immediate blocker.

## Confirmed by device evidence

| Area                     | Observation                                                                                                                  | Evidence label                 |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| Device identity          | MODEL=SM-S721B , device r12s , platform erd9945 , hardware s5e9945 .                                                         | Confirmed                      |
| SGPU node                | /dev/dri/renderD128 exists and is bound to /sys/bus/platform/drivers/sgpu .                                                  | Confirmed                      |
| Display DRM              | /dev/dri/renderD129 is bound to exynos-drm ; it is distinct from the SGPU render node.                                       | Confirmed                      |
| Device Tree              | OF_COMPATIBLE_0=samsung-sgpu , samsung-sgpu , full name /sgpu@22200000 .                                                     | Confirmed                      |
| ASIC report              | device_id=0x000073a0 , chip_rev=0x02600200 , family 147 (M6FX) , GEN=2 , MOD=0x60 , EVT=2 .                                  | Confirmed                      |
| Compute/GFX IP           | One GFX IP and one COMPUTE IP, both version 10.0 ; DMA count was zero in the probe output.                                   | Confirmed                      |
| Execution width/topology | SE=1 , SA/SE=2 , CU_active=12 , CU/SH=6 , wave=32 , RB=4 .                                                                   | Confirmed for this capture     |
| Addressing               | VA offset 0x8000000 , maximum 0x800000000000 , alignment 0x1000 , GART page size 0x1000 .                                    | Confirmed for this capture     |
| Firmware                 | SGPU 2.23.0 , RTL CL 0x0004ea15 ; ME 0x5 , MEC 0x4 , PFP 0x7 , RLC 0x1 ; CE/MC/SDMA/SMC and RLC subcomponents reported zero. | Confirmed for this capture     |
| Page faults              | faults=0 at the time of the probe.                                                                                           | Confirmed for this capture     |
| Memory/VM probe          | GEM_CREATE , CPU mmap+touch , VA_MAP , VA_UNMAP , and GEM_CLOSE succeeded.                                                   | Confirmed for this capture     |
| Failed queries           | DRM version failed with errno=14 ; SGPU_KMD_VERSION failed with errno=22 .                                                   | Confirmed for this capture     |
| Vulkan loader            | System loader path /system/lib64/libvulkan.so , observed size 240,208 bytes.                                                 | Confirmed by report            |
| Vendor ICD               | /vendor/lib64/hw/vulkan.samsung.so , observed size 44,423,944 bytes.                                                         | Confirmed by report            |
| Vulkan namespace result  | Only llvmpipe was visible through the Termux-accessible loader; root plus temporary Permissive did not change it.            | Confirmed for this environment |
| Direct ICD attempt       | Linker namespace denied /vendor/lib64/hw ; the process then segfaulted.                                                      | Confirmed for this approach    |
| Security state           | SELinux was restored to Enforcing after the experiment.                                                                      | Confirmed by logs              |

## Source-backed architecture

The Samsung source release contains kernel/include/uapi/drm/sgpu\_drm.h , a Samsung SGPU driver under kernel/drivers/gpu/drm/samsung/gpu/sgpu , the Samsung platform Device Tree files s5e9945-sgpu\_common.dtsi and s5e9945-sgpu\_evt0.dtsi , Samsung IOMMU support, Samsung DMA-BUF heaps, and GPU register headers under include/asic\_reg .

The UAPI header is AMDGPU-derived in naming and layout. It defines GEM creation, GEM mmap, contexts, BO lists, command submission, information queries, GEM VA operations, waits, VM, scheduler, and Samsung-specific instance/memory-profile operations. It defines GFX, COMPUTE, and DMA IP types; command-stream chunks for IBs, fences, dependencies, sync objects, BO handles, and timeline operations; and queries for device information, firmware, memory, registers, sensors, KMD version, and GPU page faults.

The source Makefile shows that the `sgpu` module compiles broad AMDGPU-derived subsystems, including `amdgpu_gem`, `amdgpu_cs`, `amdgpu_vm`, `amdgpu_ring`, `amdgpu_sync`, `amdgpu_sched`, GMC/MMHUB/GFX/SDMA blocks, firmware handling, Samsung DVFS/AFM/IFPO/debug/profiler pieces, and tracepoints. This is evidence of implementation material in the source release. It is not evidence that a user-space Mesa driver can use the ABI without understanding the Samsung modifications and runtime contract.

The s5e9945 Device Tree declares `sgpu@22200000`, compatible `samsung-sgpu`, `samsung-sgpu`, six register regions named `gpu`, `doorbell`, `debug`, `pwrctl`, `sysreg`, and `htu`, SGPU and GPU-AFM interrupts, `CHIP_VANG06H_LITE`, a GPU power domain, and `dma-coherent`. The EVT0 include sets `chip_revision = <0x02600100>`; the observed device reports `0x02600200`, so revision-specific data must not be merged without an explicit correlation.

## Test classification

| Artifact                                                         | What actually happened                                                                                                                                     | Correct class                             |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>sgpu_raw_probe.c</code>                                    | Read-only DRM/KMD queries plus reversible 64 KiB BO/CPU map/VA map/unmap; no submission.                                                                   | Bring-up probe and memory/VM smoke test   |
| <code>probe_SM-S721B.txt</code>                                  | Recorded identity, IP, firmware, page faults, and BO/VA results.                                                                                           | Raw probe output                          |
| <code>vk_exec_props.c</code>                                     | Loader/instance/device selection, trivial pipeline creation, executable property/statistics/IR queries; no queue retrieval, dispatch, submit, or readback. | Vulkan pipeline metadata probe            |
| <code>exec_props.txt</code> and <code>exec_props_root.txt</code> | Both report one <code>llvmpipe</code> device and no visible Samsung GPU.                                                                                   | Negative loader observation               |
| Direct ICD experiment                                            | <code>dlopen</code> path blocked by linker namespace and followed by SIGSEGV.                                                                              | Discarded approach for this namespace     |
| <code>FRIEND_PROBE.md</code>                                     | Instructions and expected outputs, including a statement that no GPU work is submitted.                                                                    | Test plan/initializer guidance            |
| <code>vulkan.samsung.so</code> and <code>libdrm_sgpu.so</code>   | Vendor binaries copied for offline analysis.                                                                                                               | Reference binaries; not open project code |
| Samsung source archive                                           | Source and build material available for inventory.                                                                                                         | Source evidence; license review required  |

## Negative results that remain useful

The two SGPU probe failures are preserved: DRM version `errno=14` and `SGPU_KMD_VERSION errno=22`. They close only those query paths under that exact invocation. They do not prove that DRM, KMD, or the GPU is unusable because the same run successfully returned device/IP/firmware/page-fault information and completed memory/VM operations.

The Vulkan result closes one specific path: the loader visible to the Termux process, even with root and temporary permissive SELinux, did not expose the Samsung device. It does not prove that the Samsung ICD is absent, that a production graphics process cannot load it, or that an external loader can never access it.

## Immediate next experiments

The safest next sequence is to preserve and hash all raw artifacts; inspect ICD dependencies and exported interfaces without loading it directly; identify the Android process, loader path, and namespace that successfully use the vendor ICD; and document the relationship between the public-looking UAPI and the Samsung driver implementation. Only after the Samsung device is genuinely visible to a supported loader should layers or a real queue/compute experiment be attempted.

A real compute milestone requires all of the following in one report: target-device proof, resource allocation and mapping, queue selection, command recording, queue submission, synchronization, and independent validation of the expected result. Until then, the project must not claim compute execution, ISA extraction, a compiler backend, or an independent Vulkan driver.

## References

### Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5_REAL_CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x40000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

---

## Capítulo incorporado: new-results-analysis.md

### Análise do pacote `5doismeiosresultadosobtidos.zip`

**Hash SHA-256:** `59ac6221373572024c0967d642f9ead1f970c6791ff2428cb5c49ae73a5fe7fc` **Tamanho comprimido:** aproximadamente 14 MiB **Data de análise:** 05 de setembro de 2026

O pacote foi lido e extraído fora do Git. O binário proprietário `vulkan.samsung.so` não foi copiado para o repositório. O relatório abaixo registra somente metadados, caminhos e conclusões derivadas.

## Conteúdo recebido

| Arquivo                                 | Tratamento                                               | Valor da evidência                                                                                                                                                  |
|-----------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| X0940_ETAPA1_DEFINITIVO.txt             | Lido integralmente por seções                            | Identificação inicial do ICD, SurfaceFlinger, namespaces e nós DRM; a primeira consulta de PID sofreu race condition, mas a captura adicional trouxe mapas válidos. |
| X0940_CHECKPOINT_ETAPA_2_DEFINITIVO.txt | Lido integralmente                                       | Fecha o mapeamento card0 → sgpu , card1 → exynos-drm , renderD128 e renderD129 .                                                                                    |
| ETAPA_X_IDENTIFICACAO_SGPU.txt          | Lido integralmente                                       | Confirma SGPU, 22200000.sgpu , firmware 2.23.0, RTL e frequências/devfreq.                                                                                          |
| ETAPA_3_COMPLETA.txt                    | Normalizado e analisado por seções e linhas de evidência | Coleta ampla de SurfaceFlinger, DRM, processos, bibliotecas e atividade gráfica; contém muito log incidental e não deve ser tratado como um único teste.            |
| Etapa_4_SGPU_Xclipse940_LOGS.zip        | Extraído e lido                                          | Identifica os processos reais do Photo Remaster, bibliotecas Vulkan/OpenCL, FDs SGPU e limites da prova de dispatch.                                                |
| v4.txt                                  | Lido integralmente                                       | Lista APIs e símbolos de libdrm/SGPU, OpenCL e Vulkan; é uma etapa de preparação e inventário, não um trace de submissão.                                           |
| Etapa 5.txt                             | Lido integralmente                                       | Registra strings, offsets, disassembly AArch64, referências de opcode/encoding e as correções contra interpretações excessivas.                                     |
| Etapa 5 documentada.pdf                 | Convertido para texto e confrontado com Etapa 5.txt      | Confirma a conclusão graduada: infraestrutura estática forte, sem prova de ISA nativa em runtime.                                                                   |
| vulkan.samsung.so                       | Mantido fora do Git; apenas metadados locais             | ELF AArch64 stripped, 44.423.944 bytes, BuildID 7b6134ba45f561f28b006e07ef075b4d4c429bcd .                                                                          |

## Conclusões promovidas

### Loader e caminho de produção

A coleta válida do SurfaceFlinger mostrou vulkan.samsung.so e /system/lib64/libvulkan.so mapeadas no processo, além de FDs para /dev/dri/renderD128 . A diferença entre o mount namespace do SurfaceFlinger e o do Termux explica por que o resultado de llvmpipe no Termux não invalida a presença do ICD Samsung no sistema. O Item 1 foi promovido para **caminho de produção confirmado**, mas a enumeração Vulkan detalhada ainda não foi capturada.

## SGPU e DRM

O mapeamento `card0 → sgpu` e `renderD128` foi confirmado com sysfs, Device Tree e driver. `card1 → exynos-drm` e `renderD129` permanecem separados. A identidade de plataforma `/sgpu@22200000`, o driver `/sys/bus/platform/drivers/sgpu`, firmware SGPU `2.23.0`, RTL `0x0004ea15` e frequências devfreq foram registrados.

## OpenCL e Photo Remaster

O processo correto de serviço foi corrigido para `com.samsung.android.photoremasterservice:photoremasterservice`. Esse processo carregou simultaneamente `vulkan.samsung.so`, `libdrm_sgpu.so`, `libOpenCL.so` e `libSGPUOpenCL.so`, mantendo FDs em `renderD128`. A API OpenCL expõe contexto, fila, buffers, kernels, `clEnqueueNDRangeKernel`, sincronização e readback. Isso confirma um caminho real de compute do vendor, mas não é prova de um kernel próprio concluído nem de readback controlado pelo projeto.

## Compiler e encoding

A análise do ICD encontrou strings e contexto relacionados a SPIR-V, shader compiler, opcode translation, hardware opcode, emitter, encoder, VOPC/VOP1/VOP2/VOP3/VOP3P e DPP8/DPP16. A análise também mostrou que referências individuais como `AMD Shader Compiler`, `gfx10_4_GEN`, `SCAsmEncoder.cpp` e `SCemitV0p3` podem aparecer em mensagens de log. Portanto a classificação correta é **infraestrutura estática fortemente sustentada**, não “ISA Xclipse decodificada” e não “backend runtime comprovado”.

## Negativas importantes

O pacote não fornece um trace comprovando a cadeia completa `B0 antes → submit real → execução GPU → fence → B0 depois/readback → recuperação`. A presença de `sgpu_cs_submit`, `amdgpu_cs_submit`, `clEnqueueNDRangeKernel` ou `clEnqueueReadBuffer` como símbolos não altera essa negativa. Também não fornece uma relação runtime entre shader SPIR-V controlado, binary gerado e instrução ISA nativa da Xclipse 940.

## Política de publicação

O binário `vulkan.samsung.so`, bibliotecas vendor e logs crus permanecem fora do Git. O repositório publica apenas conclusões, metadados, caminhos, hashes e documentação de proveniência. Qualquer código Samsung continua sujeito a revisão de licença e não deve ser redistribuído por estar presente no pacote de trabalho.

## Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de `BO_LIST` e criação de contexto. Em `A1.5-REAL-CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

---

# Capítulo incorporado: npu-enn-rootless-2026-09-18.md

## NPU/ENN rootless — atualização de 18/09/2026

### Resultado principal

A nova coleta comprovou que um processo Android sem root consegue obter o serviço público `android.hardware.neuralnetworks.IDevice/enn`, compilar um modelo NNAPI e executar um grafo `SOFTMAX` no ENN com saída correta.

A saída produzida foi `[0.032059, 0.087144, 0.236883, 0.643914]`, igual ao valor esperado, com diferença máxima `0.000000`. A cadeia observada foi: descoberta do dispositivo `enn`, finalização do modelo, compilação, execução e leitura do resultado.

### O que é novidade

A descoberta transforma a rota NNAPI/ENN em um caminho comprovadamente **rootless** para os grafos reproduzidos. Ela não depende de copiar bibliotecas vendor para o aplicativo e não usa acesso direto ao device node da NPU.

A nova evidência também corrige a classificação anterior de `SOFTMAX`. O benchmark antigo mostrou desempenho ruim, mas o teste novo comprovou suporte e execução correta em um grafo pequeno. `SOFTMAX` deve ser classificado como funcional, porém ainda não otimizado para workloads maiores.

### Limites preservados

`BATCH_MATMUL` continua não suportado pelo dispositivo `enn` no formato testado. A interface proprietária `vendor.samsung_slsi.hardware.enn_aidl.IEnnInterfaceAidl/default` não foi descoberta pelo teste, nem com UID comum nem com root. O endpoint `/dev/vertex10` exige permissões elevadas e não teve ioctl funcional demonstrado.

A conclusão correta é que existe uma rota pública NNAPI/ENN rootless, não que todo o decoder transformer possa ser transferido à NPU. A execução completa ainda depende de particionamento, medição e fallback.

### Próximo caminho documentado

O XOP passa a registrar a seguinte estrada:

```
Aplicativo sem root
↓
Android NNAPI: IDevice/enn
↓
HAL ENN Samsung
↓
Subgrafo aceito pelo driver
↓
Execução NPU e readback
```

A futura camada de pesquisa pode ser construída sobre essa interface pública, mantendo `FULLY_CONNECTED` INT8 e outros subgrafos aceitos no ENN, enquanto operações não suportadas permanecem na CPU ou em outro backend.

## Proveniência

Fonte: pacote Exynos\_NPU.zip obtido em 18/09/2026. SHA-256:  
873a0f8d3ff4d8e9a4168aafab0349f5becfa539cfc9b03d8a7033e3acb08af9 .

# Capítulo incorporado: rotas-e-falhas-2026-09-06.md

## Rotas de investigação e falhas observadas

### Rota prioritária

A rota prioritária começa pelo Device Tree em runtime e segue até o primeiro job GFX controlado. O trabalho deve reproduzir power-domain, MMIO, IRQ, memória reservada, DMA heap, IOMMU, BO, VM, scheduler e fence nessa ordem. Essa sequência reduz o risco de atribuir uma falha de plataforma a comandos ou ISA.

### Bloqueios atuais

| Bloqueio                                       | Evidência                                                                            | Consequência                                                        |
|------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| workload não isolado                           | RenderThread, Gralloc, Chrome e compositor aparecem nas janelas                      | não atribuir qualquer job ao cliente estudado                       |
| fence clássica indisponível em parte da coleta | eventos amdgpu_fence* aparecem como unavailable em P4.12                             | correlacionar por seqno, tracepoints disponíveis e saída controlada |
| compute não demonstrado                        | P5.04 não contém CS/scheduler/IB compute                                             | não declarar dispatch ou readback                                   |
| ABI vendor fechada                             | bibliotecas e exports estão presentes, mas dependências e namespaces são específicos | não assumir reutilização fora do Android vendor                     |
| formatos internos incompletos                  | PTE, IB e símbolos são observados sem especificação pública completa                 | manter implementação em modo experimental e reversível              |

### Rotas de estudo

- Cliente GFX mínimo:** criar um marcador de processo, identificar contexto e capturar apenas o intervalo do cliente; validar uma saída em buffer.
- Memória mínima:** reproduzir BO, map, VM update, flush e unmap sem submeter comandos complexos.
- Conclusão:** usar sched\_job , context , seqno , dma\_fence disponível e leitura de estado de ring para provar término.
- Compute:** começar por identificação OpenCL e um buffer pequeno; separar criação de contexto, compilação, dispatch e readback.
- Vulkan:** somente depois do contrato de memória e submissão; usar a pilha vendor como referência observacional, não como ABI aberta.

### O que deve ser evitado

Não publicar scripts de captura nem orientar a execução de probes como se fossem testes de funcionalidade. Não escrever dados arbitrários em MMIO ou firmware. Não substituir a validação de um job controlado por strings de uma biblioteca ou por exports

ELF.

## Addendum 2026-09-07 — xclipselogs

A nova rodada `xclipselogs.zip` acrescentou evidência de um cliente DRM próprio em `/dev/dri/renderD128`. O cliente confirmou abertura do render node, criação de GEM/BO, VA map/unmap, criação de BO\_LIST e criação de contexto. Em `A1.5-REAL-CS_20260907_161914`, um `DRM_IOCTL_AMDGPU_CS` foi aceito (`ioctl_ret=0`, `CS_IOCTL=ACCEPTED`) para um chunk IB (`chunk_id=0x1`) com VA `0x4000000000` e `ib_bytes=4`.

Esse resultado confirma **aceitação de uma entrada de command submission pelo KMD**, mas não confirma execução GPU, fence própria, GPU write ou readback. Variantes próximas foram rejeitadas com `EINVAL` ou `EFAULT / Bad address`. Este resultado está incorporado à documentação técnica consolidada. Fontes C, executáveis, logs crus e `dmesg` permanecem fora do repositório.

# Capítulo incorporado: source-archive-initial-analysis.md

## Initial Source Archive Analysis

**Archive:** Tudo sobre a `xclipse.zip` **SHA-256:** `cd73e2fa24ac083b9babf243b39772065b060abc9cd30fb476acabe74433e67f` **Archive members:** 17 total, 16 files **Local extraction:** área local de análise de fontes

### Scope of this pass

This pass verifies archive integrity, records provenance, classifies members by extension and filename, extracts text-search signals, and creates a license-review queue. It does not assert that a filename is a working test, that source code is redistributable, or that a vendor component can be loaded outside Android.

### Inventory summary

| Class            | Count |
|------------------|-------|
| binary           | 2     |
| document         | 4     |
| log-or-text      | 4     |
| other            | 4     |
| source-or-config | 2     |

## Extensions

| Extension           | Count |
|---------------------|-------|
| <code>.pdf</code>   | 4     |
| <code>.zip</code>   | 4     |
| <code>.txt</code>   | 3     |
| <code>.md</code>    | 2     |
| <code>.so</code>    | 2     |
| <code>[none]</code> | 1     |

## Interpretation boundary

The inventory can establish that files exist in the supplied archive and that the ZIP is readable. It cannot establish that a test executes GPU work. Any directory or executable whose name contains `test`, `probe`, `init`, or `check` must be inspected for the execution path, target-device selection, submission, synchronization, and result validation before being described as a real test.

The next step is a manual and source-aware review of the high-value paths listed in `source-analysis/keyword-index.md`, followed by per-experiment reports with raw outputs.

---

# Capítulo incorporado: test-classification.md

## Test and Probe Classification

This report is a conservative filename/content triage of the supplied archive. It is not a test result. A “possible execution test” still requires manual inspection of target-device selection, resource allocation, submission, synchronization, and validated output.

| Archive member                                                                          | Conservative class          | Cold interpretation                                                                    |
|-----------------------------------------------------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------|
| Tudo sobre a xclipse/Backup_completo_de_continuidade_-_projeto_Xclipse_.md              | continuity report           | Narrative and plans; not a runtime result by itself.                                   |
| Tudo sobre a xclipse/Relatório_técnico_-_caminho_do_ICD_Vulkan_Xclipse_.md              | loader/ICD report           | Interpretation of loader observations; not a driver test.                              |
| Tudo sobre a xclipse/SAVE_retomada_caminho_implementação Vulkan de referência.pdf       | continuity PDF              | Plans, commands, and conclusions requiring artifact cross-check.                       |
| Tudo sobre a xclipse/SM-S721B.zip                                                       | source/device package       | Contains Samsung source and platform material; not a test.                             |
| Tudo sobre a xclipse/XCLIPSE940_PROBE_FINAL.zip                                         | probe package               | Contains initializers, raw outputs, and source; execution status depends on each file. |
| Tudo sobre a xclipse/XCLIPSE940_problemas_etapa3_em_diante.pdf                          | failure report              | Documents failures and limits; not proof of successful execution.                      |
| Tudo sobre a xclipse/backup_completo_continuidade.pdf                                   | continuity PDF              | Narrative and plans; not a runtime result by itself.                                   |
| Tudo sobre a xclipse/caminho_implementação Vulkan de referência.zip                     | probe/report package        | Contains a probe explicitly described as not submitting GPU work.                      |
| Tudo sobre a xclipse/exec_props.txt                                                     | negative Vulkan observation | One llvmpipe device; no Samsung device visible.                                        |
| Tudo sobre a xclipse/exec_props_root.txt                                                | negative Vulkan observation | Root-session repeat; no Samsung device visible.                                        |
| Tudo sobre a xclipse/libdrm_sgpu.so                                                     | reference vendor binary     | Exported AMDGPU-like symbols; not evidence that the probe executed GPU work.           |
| Tudo sobre a xclipse/logs para tentativa implementação Vulkan de referência_xclipse.zip | raw device/log package      | Environment and diagnostic outputs; classify each command separately.                  |
| Tudo sobre a xclipse/probe_SM-S721B.txt                                                 | raw SGPU probe output       | Confirms queries and memory/VM smoke path; no submission.                              |
| Tudo sobre a xclipse/relatorio_tecnico_icd_vulkan.pdf                                   | loader/ICD report           | Interpretation of observations; not an execution test.                                 |
| Tudo sobre a xclipse/sgpu_raw_probe                                                     | AArch64 probe binary        | Its source explicitly submits nothing; classify as bring-up                            |

| Archive member                         | Conservative class      | Cold interpretation                                                       |
|----------------------------------------|-------------------------|---------------------------------------------------------------------------|
|                                        |                         | probe.                                                                    |
| Tudo sobre a xclipse/vulkan.samsung.so | reference vendor binary | Present on device/archive; not evidence of loader usability or execution. |

### Promotion rule

No row is promoted to a real execution test from a name, a successful build, a test framework assertion, or an API return code alone. Promotion requires an evidence report with raw output and an end-to-end result.

## Fechamento: o que está efetivamente comprovado

A plataforma possui Device Tree observável para SGPU/G3D e NPU, nós DRM e infraestrutura de memória/VM/scheduler no caminho Android suportado. A pilha vendor possui bibliotecas Vulkan e OpenCL instaladas, mas o carregamento externo e o dispatch compute independente não foram comprovados. A execução SGPU com resultado controlado continua distinta de enumeração, submissão aceita ou atividade de scheduler.

No ramo NPU, a cadeia pública NNAPI → dispositivo enn → compilação → execução → readback foi comprovada sem root para o grafo SOFTMAX reproduzido. O endpoint direto /dev/vertex10 não foi liberado para UID comum; o serviço AIDL vendor direto não foi encontrado pelos probes; e os ioctl's privilegiados testados foram inconclusivos. Os benchmarks mostram vantagem apenas em workloads específicos, como o FULLY\_CONNECTED INT8 de quatro camadas, e CPU otimizada venceu em outros workloads.

Os dados brutos, logs integrais, dumps, bibliotecas vendor, firmware, binários de coleta, credenciais, modelos proprietários e identificadores pessoais não fazem parte deste documento público. O conteúdo abaixo é a consolidação técnica derivada e sanitizada do que foi descoberto.